Sharing a Secret Anamorphically: Secret Shares Dressed Up as Signatures

Gennaro Avitabile[1], Vincenzo Botta[2], and Daniele Friolo[2]

[1] IMDEA Software Institute, Madrid, Spain
avitabilegenn@gmail.com
[2] Sapienza University of Rome, Rome, Italy
botta.vin@gmail.com
friolo@di.uniroma1.it

Abstract. Anamorphic Encryption (Persiano, Phan and Yung, Eurocrypt '22) allows private communication in a challenging setting where encryption is severely controlled by a central authority (henceforth the dictator) who can demand the users to surrender their secret keys. Anamorphic Signatures (AS) (Kutylowski, Persiano, Phan, Yung, Zawada, Crypto '23) face the even more restrictive world where only authentication is allowed but users still want to send secret messages despite the repressive control of the dictator holding their signing key. Several constructions have been proposed so far, but they all come with some limitations.
We propose a new flexible setting where digital signatures are used to covertly embed secret shares which t out of N designated receivers can combine to recover a covert secret. We formalize this new primitive called Anamorphic Secret Sharing Signatures (ASSS) together with new robustness, forward secrecy, and private unforgeability notions and we give a construction for Schnorr-like signatures overcoming the limitations of previous constructions. ASSS additionally imply (regular single-receiver) AS, but two signatures (instead of one) are needed to recover the covert message.

1 Introduction

Anamorphic Encryption (AE) [41] enables private communication in environments where encryption is tightly regulated by a central authority. AE allows a sender to covertly transmit a confidential message to a receiver without raising suspicion, even if the authority has the receiver's secret decryption key. This concept is particularly relevant in dictatorships, where encryption may remain legally permitted to protect against external threats, yet individuals can be compelled to hand over their private keys. In [29] an even stricter scenario is considered where the dictator forbids encryption altogether and only allows authentication via the use of digital signatures. This stronger scenario automatically relieves the dictator from the problem of covert communication via AE, but the possibility of exchanging secret messages in spite of the repressive control of the dictator is still opened up by the presence of anamorphic signatures.

Anamorphic Signatures (AS) operate under two distinct modes: standard and anamorphic. In the standard mode, AS behave like any typical signature scheme. In anamorphic mode, however, key generation produces a public key pk along with two secret keys: a conventional signing key sk and a "double key" dk. Alice shares dk with Bob through a private one-time channel, while using pk publicly. If compelled to reveal her secret key, Alice can safely disclose only sk. Alice, who knows dk and sk, can craft a signature that both authenticates a message m under her public key and simultaneously encodes a hidden message $\hat{m}$, which only Bob can extract using dk. The core security goal is that the anamorphic mode is indistinguishable from the standard mode, even when the dictator knows sk. Schemes that additionally allow the double key to be sampled independently of the conventional keys are called Anamorphic Extensions [4]. As noted in [41], designing entirely new schemes that are compatible with anamorphic communication offers limited benefit. A dictator could simply mandate any scheme they prefer, potentially to suppress anamorphic messaging. Consequently, several works proved that existing encryption

schemes are inherently anamorphic [41, 4, 13, 30, 42]. Many of these constructions are not only efficient but also have an exponentially large anamorphic message space. Analogously, recent works [29, 3] have shown that several existing signature schemes are anamorphic, unfortunately such schemes come with some limitations that we discuss next.

On unique signatures. The most effective way for a dictator to suppress anamorphic communication is through unique signature schemes. In such schemes, each message can only produce a single valid signature under a given key, eliminating the possibility of embedding covert information. However, practical adoption may be hindered by compatibility requirements with existing systems (e.g., Bitcoin). This trade-off often dictates the choice of cryptographic primitives. For instance, Schnorr multi-signatures [27, 38, 2], despite notable drawbacks compared to BLS [11], have gained significant traction precisely because of their compatibility with standard Schnorr signatures.

Constructing AS. In [29] various techniques for AS are presented. Among those, rejection sampling is a technique for getting an AS extension on top of any signature scheme whose signatures have high min-entropy. The double key is simply a PRF key K and the sender samples a new signature σ until $\mathsf{PRF}(K, \sigma)$ is equal to the desired message. However, if the signer wants to embed a k-bit message, 2^k samples are necessary on average and thus, to keep signing in polynomial time, the size of the message must be logarithmic.

It is instead more interesting to look for AS with an exponentially large anamorphic message space. In [29] they give a sufficient condition for anamorphism of signature schemes called randomness recoverability. That is, a signature scheme is randomness recoverable if it allows, with sufficient knowledge (e.g., with the secret key), to recover from the signature (a part of) the random coins used during the signing process. Given a randomness-recoverable signature scheme it is straightforward to encrypt an anamorphic message $\hat{m}$. The signer encrypts $\hat{m}$ using a symmetric encryption scheme with pseudo-random ciphertexts and uses the obtained ciphertext as the random coins of the signing process. The double key simply consists of the secret key of the symmetric encryption scheme and all the information that allows to recover the randomness. This technique gives an exponential anamorphic message space.

Based on this insight, we can then divide signature schemes in different categories depending on what information is necessary to recover the signing randomness:

Recovering the randomness requires no secret: Clearly randomness-recoverable signatures are schemes directly including uniformly random bit-strings in $\{0,1\}^{O(\lambda)}$ in the signature itself. For example, in [29] the Boneh-Boyen [8], the RSA-PSS signatures [6], and the Naor-Yung transform [37] applied to the Lamport tagging scheme [31] - three schemes with such a property - are shown to be anamorphic. There are many other, widely known, schemes with such a property (e.g., [9, 26, 7]). Although the techniques shown in [29] to embed anamorphic messages in such signature schemes are elegant and sophisticated, the schemes they are built on top quite clearly embed a steganographic channel and are easily forbidden by a dictator eager to prevent covert communication.

Recovering the randomness requires the secret key: Other constructions are randomness recoverable only given the signing secret key. This constraint has two disadvantages: (i) the sender has to trust the anamorphic receivers, as they can sign messages on her behalf using the signing secret key, even though the sender did not ever send an anamorphic message and (ii) every time the sender has to update her public key the demanding secure exchange of the double key (e.g., in person) must be repeated to enable anamorphic communication. For example, in [29] the ElGamal [22, 43] and the Schnorr [44] signatures are shown to be randomness recoverable, but only with the help of the signing key. For example, the Schnorr

scheme includes a uniformly random group element a in the signature, but it cannot be directly used to embed a pseudo-random ciphertext since the signer needs to know the discrete log of a to finalize the signature.

Currently, apart from schemes that embed uniformly random bit-strings directly into the signature, there are basically no AS constructions, for widely used schemes, providing an exponentially large anamorphic message space while allowing anamorphic channel setup without revealing the secret key to the receiver. The recent work of Deo and Libert [16] proposes a remarkable bandwidth-efficient construction for the RSA-based Guillou-Quisquater scheme. However, such scheme is not very used in practice. Instead, the concretely more efficient dlog-based protocols are now widely deployed and used across numerous applications. The same authors [16] further propose a dlog-based high-rate anamorphic signature over the Okamoto-Schnorr signature scheme [40]. However, their solution needs the users to define some public parameters together with a trapdoor, which is used to create anamorphic signatures. Realistically, a dictator would choose such public parameters (as a system manager could do according to [40]) and force the users to use them; thus making the technique of [16] unusable. More details are given in Sec. 2.

In the following section, we present a new setting that both introduces additional features and overcomes the limitations on a class of widely used schemes, including the Schnorr signature in the dlog setting, albeit at the cost of requiring two signatures (rather than one) to recover the anamorphic message.

1.1 Anamorphic Secret Sharing Signatures

We study the scenario of a dealer D who privately sends double keys $\mathsf{dk}_1, \ldots, \mathsf{dk}_N$, sampled *independently from the signing key*, to N different receivers $R_1, \ldots, R_N$, and anamorphically encodes secret shares of an anamorphic secret $\hat{s}$ in digital signatures $\sigma_1, \ldots, \sigma_n$ of unrelated regular messages $m_1, \ldots, m_n$.

Using the double key dk_i and the signature σ_i, R_i can extract a share from the signature. Less than t shares computationally hide $\hat{s}$, while t or more shares allow to recover $\hat{s}$. We call this primitive Anamorphic Secret Sharing Signature (ASSS). As in AS, we require that regular signatures are indistinguishable from signatures anamorphically encoding shares.

ASSS enrich the functionality of AS ensuring that the anamorphic message can only be reconstructed by a quorum of receivers. In a dictatorship, anamorphic communication can be seen as a last resort to communicate and coordinate within groups despite the repressive regime. For example, the dealer could be a relevant party (e.g., another authority close to the dictator) who wants to send signatures[3] to whistleblow secrets to a series of parties (e.g, the press) with the guarantee that a minimum number of parties will learn the leaked information about the dictator.

In such a scenario, we require ASSS to satisfy some additional properties w.r.t. AS. First, the ASSS should have a form of *robustness*. Indeed, we support the even more repressive setting where only the dealer is allowed to generate signatures and thus reconstruction must happen in person[4]. Therefore, it is crucial that the receivers are aware of whether a signature encodes a share or not, as just *attempting* a reconstruction that it is going to fail might be wasteful or even dangerous. This notion of robustness is different from the one defined in [4] (see Sec. 1.3 for more details). Second, we require some form of *forward secrecy*, meaning that a single demanding[5] setup phase of the double key is sufficient to share an unbounded number of secrets;

[3] These signatures might be involved in her normal activity and even be posted on a public bulletin board.

[4] If everyone is allowed to generate signatures, the parties could just send each other the shares using anamorphic signatures. Indeed, in the next paragraph we show that ASSS imply a form of AS.

[5] The dealer may be a relevant party who is closely watched by the dictator.

and a successful reconstruction phase does not impact on the privacy of the subsequent shared anamorphic messages. Lastly, we require some form of *recipient unforgeability* meaning that the signature remains unforgeable by the receiver (i.e., the signing key is not leaked to the receivers) until a reconstruction is performed. Recipient unforgeability becomes very handy when the creation of the double key is independent from the creation of the signing key. Indeed, after having shared a secret, the dealer can ask for a new signing key[6] avoiding impersonation attacks without losing her ability to communicate anamorphically.

ASSS with one receiver. We remark that ASSS are useful even for one-to-one communication. Indeed, the dealer can make a 2-out-of-2 secret sharing giving both dk_1 and dk_2 to the same receiver. In this way, the receiver can recover the anamorphic message after having received 2 signatures.

This is similar to the notion of anamorphic signature multi-message extensions recently proposed in [3]. However, their constructions do not overcome the limitation we highlighted in Sec. 1 (see Sec. 2 for more details).

If the ASSS has good bandwidth, this solution is preferable to a regular AS with a small message space (e.g., via rejection sampling), as it would require much more signatures to be sent to convey the same amount of information.

1.2 Our Contribution

We construct a class of ASSS schemes based on the Fiat-Shamir transform of a class of Σ-protocols called pre-image Σ-protocols [34, 15, 14], which include the widely used Schnorr identification scheme.

Our scheme has the following features:

1. The generation of the double keys is independent of the generation of the signing key. Therefore, the sender can generate a signing key with the regular key generation procedure and can re-use the same double keys even if he needs to update his signing key.
2. The anamorphic message space is exponentially large.
3. It enjoys the robustness, forward secrecy, and recipient unforgeability explained in Sec. 1.1.

We remark that the previous construction of AS for Schnorr [29] does not enjoy recipient unforgeability, as the double key simply consists of the signing secret key. In this sense, before our results, a dictator could have felt comfortable with allowing Schnorr signatures since the only known way to make them randomness recoverable, and thus to encrypt big anamorphic messages, was to give the secret key to the receiver even if no anamorphic message would ever be encrypted in the future. This is a big disincentive to anamorphic communication if the ability to sign messages is valuable, e.g. it allows to spend money on the dictatorship's digital currency.

In our setting, instead, even if the signing key gets leaked once the anamorphic communication takes place, the sender can simply perform a key revocation procedure and get a new signing key. This is without the need to do a fresh (demanding and possibly risky) exchange of a double key shall further anamorphic communications take place.

1.3 Technical Overview

Let us consider the Schnorr Σ-Protocol for proving knowledge of a discrete log. The works of Cramer [14], Cramer and Damgard [15], and Maurer [34] showed that a protocol (called Pre-Image Σ-Protocol) for proving knowledge of a pre-image of a group homomorphism generalizes

[6] A key revocation procedure should always be made available since the risk of compromising secret keys does not disappear if a dictatorship takes place.

a large number of Σ-protocols in the literature including the Schnorr's protocol [44] and the Guillou-Quisquater protocol [24]. For simplicity we will now focus on the Schnorr Σ-Protocol, but our techniques apply to the pre-image Σ-protocols.

In the Schnorr Σ-protocol, the common input consists of a group $\mathbb{G}$ of a prime order p, a generator g and $x \in \mathbb{G}$. The prover knows w such that $x = g^w$. The protocol proceeds in three rounds: (i) the prover sends to the verifier a value $a = g^r$ where r is sampled at random, (ii) the verifier sends to the prover a uniformly random challenge c, (iii) the prover replies with $z = r + cw$. Finally, the verifier accepts iff $g^z = a \cdot x^c$. In [29] it is observed that Schnorr (as many other Σ-Protocols) is randomness recoverable if the witness w is known. Indeed, the randomness r used to produce first-round message a can be obtained from z as $r = z - cw$. Such strategy works identically if the Σ-protocol is turned into a signature using the Fiat-Shamir transform.

We instead exploit another property of Σ-protocols to extract the randomness in a different way without being directly provided with the witness. That is, Σ-protocols satisfy a property called special soundness stating that from any two accepting transcripts (a_0, c_0, z_0) and (a_1, c_1, z_1) for a statement x such that $a_0 = a_1$ and $c_0 \neq c_1$, one can extract the witness w. For example, the extractor of Schnorr computes the witness as $w = (z_0 - z_1)(c_0 - c_1)^{-1}$.

A basic idea for a 2-out-of-2 anamorphic secret sharing protocol involves sending to two receivers two accepting transcripts sharing the same a so that they can put together the transcripts to recover w and hence r. However, aside being insecure, producing two transcripts/signatures with the same a makes anamorphic communication easily detectable by the dictator! Furthermore, this approach does not generalize to arbitrary values of t and N.

Adding anamorphic security. To prevent the dictator from detecting anamorphic communication we exploit the homomorphic properties of Schnorr i.e., that $g^{r_1} \cdot g^{r_2} = g^{r_1 + r_2}$. A 2-out-of-2 anamorphic secret sharing protocol consists in generating two PRF keys K_0 and K_1 as double keys and to share a synchronized counter ctr with the receivers. To encode an anamorphic share of a secret $\hat{s}$ for the i-th receiver with $i \in \{0, 1\}$, the dealer produces as a first round message $a_i = g^{r_i}$ where $r_i = \hat{s} + \mathsf{PRF}(K_i, ctr)$. The receivers can both compute the same first round message $a = a_i \cdot g^{-\mathsf{PRF}(K_i, ctr)}$ and consequently adapt their z_i as $z_i' = z_i - \mathsf{PRF}(K_i, ctr)$. They can then run the extractor algorithm on the transcripts (a, c_0, z_0') and (a, c_1, z_1') to get w and consequently $r_i - \mathsf{PRF}(K_i, ctr) = \hat{s}$.

Generalizing to arbitrary thresholds. One can easily extend the above template to N receivers, but this comes with the inconvenience that the reconstruction threshold is fixed to $t = 2$. Indeed, any two receivers can get the same first-round message and come together to run the extractor. Our approach to overcome this limitation is having the dealer producing the first-round messages a_i with $i \in [N]$ in a way that t receivers must cooperate to unblind their a_i and get the same a. After this unblinding procedure, any two transcripts can be used to run the Σ-protocol's extractor and recover the anamorphic message $\hat{s}$.

To achieve this goal we use distributed pseudorandom functions (DPRF) [35, 10, 36, 32]. In DPRF, a PRF secret key SK is secret shared among N parties so that each party can locally compute a partial evaluation of the PRF on some input X. Given t partial evaluations it is possible to reconstruct the evaluation $\mathsf{PRF}(SK, X)$ of the PRF under the initial secret key. Roughly, the security of the DPRF mandates that, under possibly adaptive corruptions, less than t partial evaluations computationally leak nothing about the evaluation of the PRF using the initial secret key SK.

The modified protocol works as follows, the dealer generates a DPRF secret key SK_0 and shares it into $SK_1, \ldots, SK_n$. The double key of the receiver $i \in [N]$ is $\mathsf{dk}_i = SK_i$. When generating a share for the message $\hat{s}$ for the receiver i, the dealer produces $a_i = g^{r_i}$ with $r_i = \hat{s} + \mathsf{PRF}(SK_0, ctr||i)$. During the reconstruction phase, each party i provides its transcript

along with the set of their partial evaluations $\{\mathsf{PRF}(SK_i, ctr||j)\}_{j\in[N]}$. From t of these sets it is possible to reconstruct $\mathsf{PRF}(SK_0, ctr||j)$ and $\mathsf{PRF}(SK_0, ctr||k)$ for some $j \neq k \in [N]$ and thus obtain two transcripts to run the extraction procedure and recover $\hat{s}$. Notice that the dictator does not catch the dealer because every party gets a different a_i and subsequent interactions with the same receiver j also produce a random looking a_j since the counter ctr gets incremented at each interaction. Furthermore, *forward secrecy* is guaranteed because the DPRF security guarantees that an evaluation on point X tells nothing about the evaluation on another point $Y \neq X$. Unlike the constructions in [29], our construction requires parties to keep a small state, however it comes with additional properties and features.

Adding robustness. In [4] and [25] the notion of robustness for anamorphic encryption and signatures respectively is proposed. In a nutshell, it requires that ciphertexts/signatures generated using the regular scheme do not accidentally decrypt to an incorrect anamorphic message. This property is readily satisfied by our scheme since unblinding two regular first-round messages and getting the same value only happens with negligible probability. However, we ask for a somewhat stricter requirement that is parties should not even *attempt* to reconstruct a secret if no secret was encoded by the dealer. This is motivated by the fact that the dealer may need to produce/distribute signatures even when no secret to be shared is available. Since the reconstruction should happen via a secure channel (e.g., in person) it might be too costly or even dangerous for the receivers to come together only to discover no secret was encoded in the signatures. We achieve this property by adding a key K of a 1-bit output PRF in the double keys. The dealer rejection samples, over a counter u, the first-round messages $a_i = g^{r_i}$ with $r_i = \hat{s} + \mathsf{PRF}(SK_0, ctr||i) + u$ until $\mathsf{PRF}(K, a_i) = 0$ if a secret is encoded; or she rejection samples over fresh random coins r_i until $\mathsf{PRF}(K, a_i) = 1$ otherwise. The receivers simply evaluate $\mathsf{PRF}(K, a)$ to check whether a signature encodes a share or not and during the reconstruction phase he looks for values of u_j and u_k such that the unbliding of a_j and a_k gives the same a. The expected number of trails for encoding a share is 2, while it is 4 for reconstructing a secret.

2 Related Work

Anamorphic Encryption is conceptually related to, though distinct from, several previously studied notions. A detailed comparison is provided in [41]. Moreover, [45, 12] established a strong connection between AE and Algorithm Substitution Attacks (ASA) [5], enabling insights from one domain to inform and reinterpret results in the other. The concept has been extended in [30, 13] to address stronger privacy guarantees, particularly in scenarios where adversaries may have access to the double key dk. The notion of robust Anamorphic Encryption (AE) was introduced in [4] and later generalized to the sender-AE setting in [45]. A further enhancement of robustness, termed anamorphic unforgeability, was proposed in [19]. The new concent of anamorphic broadcast mode along with a construction was introduced in [18]

The concept of anamorphic signatures was introduced in [29] along with several constructions. In [25] some revised definition for the security of anamorphic signatures proposed. In [3], besides other contributions for AE, the concept of anamorphic signature extensions is proposed. In anamorphic signature extensions a constant number of anamorphic signatures is used, together with the double key, to recover an anamorphic message. In particular, they provide clever asymmetric constructions (i.e., where the double key is asymmetric like in public key encryption) for Water's signatures [46] and a construction for BBS signatures [9]. The first one requires the user to fix their own parameters for the Water's hash, which the authors acknowledge to be a strong assumption in a dictatorship. The second one exploits the fact that BBS signatures directly embed a random bit-string in $\mathbb{Z}_p$ in the signature itself. The recent work of Deo and

Libert [16] proposes several anamorphic signature constructions with recipient unforgeability. One construction is based on the generalized Okamoto-Schnorr signature scheme [40] which is derived by applying the Fiat-Shamir transform to the following Σ-Protocol. The prover proves knowledge of the secret key $(x_1, \ldots, x_n)$ such as $X = \prod_{j=1}^{n} g_i^{x_i}$, where X is the corresponding public key. To make this scheme anamorphic [16] assumes the public key to be structured as $(X, g_1, \ldots, g_n)$ allowing the user to generate g_i with a trapdoor w_i s.t. $g_i = g^{w_i}$, even though this trapdoor information is not needed for regular signing. Indeed, as also stated in the original paper by Okamoto [40], the values $g_1, \ldots, g_n$ can be generated by a third party and shared by all users. Therefore, it is reasonable to think that the dictator could simply generate $g_1, \ldots, g_n$ s.t. the users do not have the trapdoor information. A second construction is based on RSA-based Guillou-Quisquater scheme however, such scheme is not very used in practice. Finally, a modification of the Lyubashevsky scheme is also proposed but it crucially requires to use much bigger system parameters than the commonly used ones.

3 Preliminaries

Given a pair of interactive Turing machines, P and V, we denote by $\mathsf{trans}(\langle \mathsf{P}(w), \mathsf{V} \rangle(x))$ the distribution of the transcript of the interaction (i.e., the set of exchanged messages) between the machines P and V with w being P's private input and x being common input. For two machines M, A, we let $\mathsf{M}^{\mathsf{A}}(x)$ denote the output of machine M on input x and given oracle access to A. We denote by $x \leftarrow\!\!\$\ \mathcal{X}$ the process of uniformly choosing an element x in a set $\mathcal{X}$. When considering a distribution D w.r.t a set $\mathcal{X}$, we denote $x \leftarrow\!\!\$\ D$ the process of sampling from the distribution D. We implicitly assume that all the distributions we consider are efficiently sampleable.

With a little abuse of notation, we assume, when needed, that a set $\mathcal{X}$ is ordered and we write $\mathcal{X}[i]$ to indicate the i-th element of $\mathcal{X}$.

When writing $b \leftarrow \mathsf{A}(x)$, we mean that b is the output of the probabilistic polynomial-time (PPT) algorithm A on input x. Furthermore, we denote with $\mathsf{A}(x; r)$ an execution of the PPT algorithm A with fixed random coins r. Any element with position (i, j) in a matrix can be accessed with $\mathsf{M}[i][j]$. The number of rows of M can be retrieved with $\mathsf{M.length}$. An entire row of elements can be added to a matrix with $\mathsf{M.append}(el_1, \ldots, el_n)$ with n the number of columns in the matrix. When, during a nested cycle, the command break is invoked, i.e. **for** $i \in [N]$ {**for** $j \in [N]${ break}}, it automatically breaks *all* the nested cycles. We use the standard notion of digital signature scheme $\mathsf{SIG} = (\mathsf{KeyGen}, \mathsf{Sign}, \mathsf{Verify})$, we refer to App. A.2 for the formal definition.

3.1 Σ-protocols

A Σ-protocol is a *3-round* public-coin protocol consisting of a tuple of PPT algorithms $(\mathsf{P}_0, \mathsf{P}_1, \mathsf{V})$. It is defined w.r.t. an NP language $\mathcal{L}$ with a poly-time relation $\mathcal{R}_{\mathcal{L}}$, a challenge space C_Σ, and a randomness space R_Σ. The algorithms work as follows:

- $a \leftarrow \mathsf{P}_0(x, w; r)$ on input statement x, witness w, randomness $r \in \mathsf{R}_\Sigma$, outputs a first-round message a.
- $z \leftarrow \mathsf{P}_1(x, w, a, c, r)$ on input statement x, witness w, first-round message a, randomness r used in the first round, and challenge $c \in \mathsf{C}_\Sigma$, outputs the third-round message z.
- $1/0 \leftarrow \mathsf{V}(x, a, c, z) = 1$, on input statement x, first-round message a, challenge $c \in \mathsf{C}_\Sigma$, outputs 1 to accept and 0 to reject.

A transcript (a, c, z) w.r.t. a statement x is called *accepting* if $\mathsf{V}(x, a, c, z) = 1$.

Throughout the we paper, we refer to $\langle \mathsf{P}(\cdot), \mathsf{V} \rangle(\cdot)$ as an execution of a Σ-protocol Σ between a prover P and a verifier V, where a is computed using P_0 and sent to V, c is sampled at random from C_Σ by V and sent to P and finally z is computed by P using P_1 and then sent to V. In this case, the output of trans will be of the form (a, c, z). We require that a Σ-protocol has the following properties: (i) completeness, meaning that an honestly generated transcript is always accepting; (ii) special soundness, meaning that given 2 accepting transcripts $(a, c_0, z_0), (a, c_1, z_1)$ for x, where $c_0 \neq c_1$ one can always efficiently extract w s.t. $(x, w) \in \mathcal{R}$; and (iii) Special HVZK (SHVZK) meaning that there exists a simulator that given x, c produces a transcript computationally indistinguishable from an honest one. The formal definitions are reported in App. A.1.

The work of [1] establishes the minimal requirement for the security of the Fiat-Shamir [23] transform when applied to three-message public-coin identification schemes. Here, the statement x is the prover's identity (e.g., a public key) and w the prover's secret key. In security against passive attacks, an impersonator tries to convince a verifier without knowing the secret key. The impersonator is further allowed to obtain a polynomial number of transcripts of honest execution of the protocol before trying to impersonate the prover. This attack is considered *passive* since the impersonator can obtain only honest transcripts. If a PPT impersonator wins this game with negligible probability, then the protocol is secure against passive attacks. We provide the formal definition in App. B.

Theorem 1 ([1]). *If a 3-round protocol* (P, V) *is secure against* passive attacks *(Def. 18), then the signature scheme obtained by applying the Fiat-Shamir heuristics is EUF-CMA (Def. 16) in the random oracle model (ROM).*

It is known that Σ-protocols for hard relations (i.e., relations for which given the statement it is hard to find the witness) are secure against passive attacks.

3.2 Pre-image Σ-protocol

Let $(\mathcal{G}, *)$ and $(\mathcal{H}, \otimes)$ be two groups with efficiently computable operations, and let $f : \mathcal{G} \to \mathcal{H}$ be a one-way homomorphism, i.e., $f(x * y) = f(x) \otimes f(y)$. The Pre-image Σ-protocol for the relation $\mathcal{R} = \{(x, w) : x = f(w)\}$ is defined as follows:

- **Common input:** Description of groups $\mathcal{G}$, $\mathcal{H}$, and element $x \in \mathcal{H}$.
- **Prover's private input:** $w \in \mathcal{G}$ such that $x = f(w)$.

The algorithms work as follows:

1. $a \leftarrow \mathsf{P}_0(x, w; k)$**:** on input $(x, w) \in \mathcal{R}$ and random coins $k \in \mathcal{G}$, compute $a \leftarrow f(k)$, and output a.
2. $z \leftarrow \mathsf{P}_1(x, w, c, k)$**:** on input $(x, w) \in \mathcal{R}$, k, and challenge $c \in \mathsf{C}_\Sigma \subseteq \mathbb{N}$, compute and output $z = k * w^c$.
3. $1/0 \leftarrow \mathsf{V}(x, a, c, z)$**:** on input (x, a, c, z), output 1 if and only if $f(z) = a \otimes x^c$.

In [34] it is proven that pre-image Σ-protocols are SHVZK. The simulator Sim, on input instance x and challenge c, operates as follows: (i) Randomly pick $z \leftarrow G$; (ii) Compute $a = f(z) \otimes x^{-c}$; (iii) Output (a, z).

Additionally, Theorem 3 of [34] describes the condition under which a pre-image Σ-protocol is special sound. That is values $\ell \in \mathbb{Z}$ and $u \in \mathcal{G}$ are known such that:

1. $gcd(c_1 - c_2, \ell) = 1$ for all $c_1, c_2 \in \mathsf{C}_\Sigma, c_1 \neq c_2$

2. $f(u) = x^\ell$.

Whenever such conditions are met, it is easy to see that the pre-image Σ-protocol also satisfies the following stronger flavor of special soundness.

Definition 1 (Special Soundness with Randomness Recoverability). *A Σ-protocol $\Sigma = (\mathsf{P}_0, \mathsf{P}_1, \mathsf{V})$ is special sound with randomness recoverability if there exists a PPT extractor ExtR, such that on input x and 2 accepting transcripts $(a, c_0, z_0), (a, c_1, z_1)$ for x, where $c_0 \neq c_1$, it holds that:*

$$\Pr[(x, w) \in \mathcal{R}_\mathcal{L} \wedge a = \mathsf{P}_0(x, w; r) | (w, r) \leftarrow \mathsf{ExtR}(x, a, c_0, z_0, c_1, z_1)] = 1.$$

Indeed, ExtR simply runs the extractor guaranteed by theorem 3 of [34] to recover w and then computes $k = z * w^{-c}$.

3.3 Pseudo-Random Functions

A pseudorandom function (PRF) family is specified by poly-time algorithms $(\mathsf{KeyGen}, \mathsf{Eval})$, where KeyGen is a randomized key generation algorithm that takes in input a security parameter 1^λ and outputs a random key $K \in \mathcal{K}$. Eval is a deterministic evaluation algorithm, which takes in input a key $K \in \mathcal{K}$ and an input X in a domain $\mathcal{X}$ and evaluates a function $\mathsf{Eval}(K, X)$ in a range $\mathcal{Y}$.

The standard security definition for PRFs requires that the adversary is unable to distinguish an oracle outputting real pseudorandom values from an oracle using a random functions. We refer to App. A.3 for the formal definition.

3.4 Distributed Pseudo-Random Functions (DPRFs)

A Distributed PRF [7] (DPRF) is a tuple of poly-time algorithms $(\mathsf{Setup}, \mathsf{KeyGen}, \mathsf{Share}, \mathsf{PEval}, \mathsf{Eval}, \mathsf{Combine})$ with the following specification.

- $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda, 1^N, 1^t, 1^\ell)$: On input a security parameter 1^λ, a number of servers 1^N, a threshold 1^t and a desired input length 1^ℓ and outputs public parameters pp, which are considered implicit input to all other algorithms.
- $K \leftarrow \mathsf{KeyGen}(1^\lambda)$: On input the security parameter 1^λ, output a key $K \in \mathcal{K}$.
- $(SK_1, \ldots, SK_n) \leftarrow \mathsf{Share}(SK_0)$: it takes as input a random master secret key $SK_0 \in \mathcal{K}$ and outputs a tuple of shares $(SK_1, \ldots, SK_N) \in \mathcal{K}^N$, which form a (t, N)-threshold secret sharing of SK_0.
- $Y_i \leftarrow \mathsf{PEval}(SK_i, X)$: On input a key share $SK_i \in \mathcal{K}$ and an input $X \in \mathcal{X}$ and outputs a partial evaluation $Y_i \in \hat{\mathcal{Y}}$.
- $Y \leftarrow \mathsf{Combine}(\mathcal{S}, \{Y_i\}_{i \in \mathcal{S}})$: On input a t-subset $\mathcal{S} \subset [N]$ together with t partial evaluations $\{Y_i\}_{i \in \mathcal{S}}$, where $Y_i \in \mathcal{Y}$ for all $i \in \mathcal{S}$, and outputs a value $Y \in \mathcal{Y}$.
- $Y \leftarrow \mathsf{Eval}(SK_0, X)$: it takes as input the master secret key $SK_0 \in \mathcal{K}$ and $X \in \mathcal{X}$ and outputs a value $Y \in \mathcal{Y}$.

Definition 2 (Consistency). *We say that DPRF is consistent if, for any $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda, 1^\ell, 1^t, 1^N)$, any master key $SK_0 \leftarrow \mathsf{KeyGen}(1^\lambda)$ shared according to $(SK_1, \ldots, SK_N) \leftarrow \mathsf{Share}(SK_0)$, any t-subset $\mathcal{S} = \{i_1, \ldots, i_t\} \subset [N]$ and any input X, and $Y_{i_j} = \mathsf{PEval}(SK_{i_j}, X)$ for each $j \in [t]$:*

$$\Pr[\mathsf{Eval}(SK_0, X) = \mathsf{Combine}(\mathcal{S}, (Y_{i_1}, \ldots, Y_{i_t}))] \geq 1 - \mathsf{negl}(\lambda).$$

[7] This definition is the real-or-random version of the adaptive find-then-guess security of [32]. The two definitions can be shown to be equivalent with a standard hybrid argument over evaluation queries with a multiplicative gap between the advantage functions.

We say that a DPRF provides adaptive security if it remains secure against an adversary that can adaptively choose which parties it wants to corrupt. In particular, the adversary can arbitrarily interleave evaluation and corruption queries as long as they do not allow it to trivially win.

Definition 3 (Adaptive Security). *We say that a* DPRF *with key space $\mathcal{K}$ input space $\mathcal{X}$, outputspace $\mathcal{Y}$ and partial output space $\hat{\mathcal{Y}}$ is pseudorandom under adaptive corruptions if for all* A*, $t, N \in \mathbb{N}, 2 \leq t \leq N$:*

$$\Pr\left[\mathsf{Exp}^{\mathsf{sec}}_{\mathsf{DPRF}}(1^\lambda, 1^t, 1^N) = 1\right] \leq \frac{1}{2} + \mathsf{negl}(\lambda)$$

where $\mathsf{Exp}^{\mathsf{sec}}_{\mathsf{DPRF}}$ *is defined in Fig. 1.*

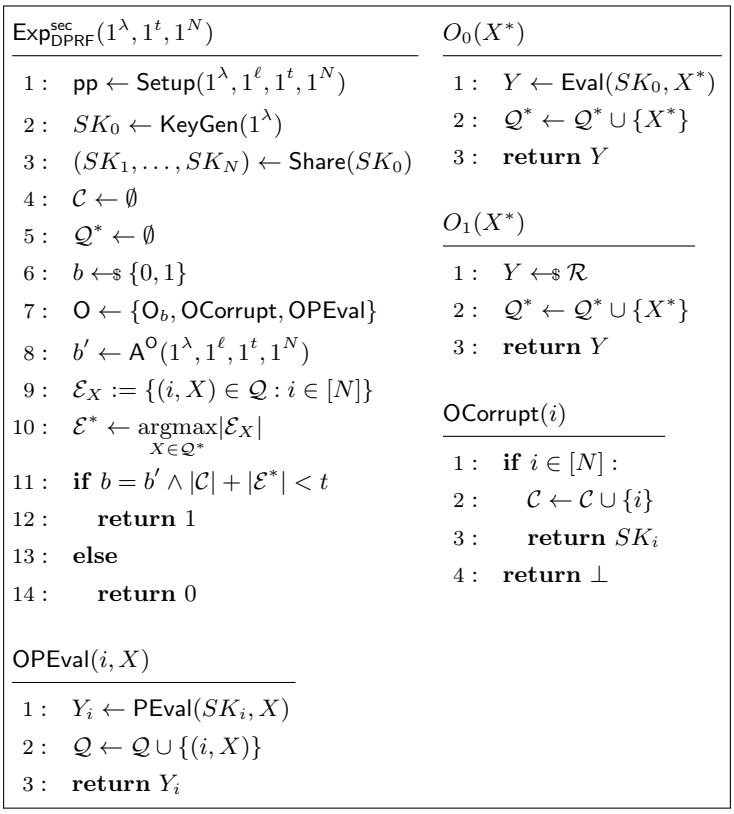

Fig. 1: Adaptive Security Game for Distributed PRFs. For any pair (i, X) in the set of queries $\mathcal{Q}$ made to OPEval, the set $\mathcal{E}_X$ contains all the queries made for different indices for a fixed $X \in \mathcal{Q}$. Then, the operator argmax outputs the set $\mathcal{E}^*$, that is the $\mathcal{E}_X$ with the greatest cardinality given that X is taken from the oracle queries in $\mathcal{Q}^*$. The check of line 10 guarantees that the adversary cannot trivially win using the corrupted keys and the partial evaluation queries.

4 Anamorphic Secret Sharing Σ-Protocol

An anamorphic secret sharing Σ-protocol $\mathsf{a}\Sigma$ is a Σ-protocol for relation $\mathcal{R}$ in which the prover and the verifier have access to an additional private input called the double key. More specifically,

the prover P wants to share an anamorphic secret $\hat{s}$ via a t-out-of-N secret sharing by talking to N verifiers $V_1, \ldots, V_N$ individually. For this purpose, P shares with each of the verifiers a specific double key, i.e. P and V_i use dk_i during their interaction. Additionally, P holds a private master double key dk.

Let $\hat{S}$ be the anamorphic secret space, R_Σ the randomness space of Σ, C_Σ the challenge space of Σ. Consider the algorithms $\mathsf{a}\Sigma = (\mathsf{dKeyGen}, \mathsf{aP}_0, \mathsf{aP}_1, \mathsf{ExtShare}, \mathsf{Recon})$ that work as follow w.r.t. a relation $\mathcal{R}$.

- $(\mathsf{dk}, \mathsf{dk}_1, \ldots, \mathsf{dk}_N) \leftarrow \mathsf{dKeyGen}(1^\lambda, 1^t, 1^N)$: on input the security parameter 1^λ, the threshold 1^t, and the number of parties 1^N , it outputs a master double key dk and individual double key for the verifiers $\mathsf{dk}_1, \ldots, \mathsf{dk}_N$.
- $a \leftarrow \mathsf{aP}_0(x, w, \mathsf{dk}, \hat{s}, \mathsf{st}; r)$ on input statement x, witness w, master double key dk, anamorphic secret $\hat{s} \in \hat{S} \cup \{\perp\}$, a state st, and random coins $r \in \mathsf{R}_\Sigma$ outputs a first-round message a.
- $z \leftarrow \mathsf{aP}_1(x, w, a, c, \mathsf{dk}, \hat{s}, \mathsf{st}, r)$: on input statement x, witness w, first-round message a, challenge $c \in \mathsf{C}_\Sigma$, master double key dk, anamorphic secret $\hat{s} \in \hat{S} \cup \{\perp\}$, state st, and randomness r used in the first round, outputs the third-round message z.
- $\mathsf{st}' \leftarrow \mathsf{UpdState}(\mathsf{st})$: on input a state st, outputs an updated state st'. The initial state is indicated with the symbol ϵ.
- $\hat{s}_i/\perp \leftarrow \mathsf{ExtShare}(x, a, c, z, \mathsf{dk}_i, \mathsf{st})$: on input statement x, first-round message a, challenge $c \in \mathsf{C}_\Sigma$, third-round message z, double key of the i-th verifier dk_i, a state st, outputs the i-th share $\hat{s}_i$, or $\perp$ if no share was encoded in the transcript.
- $\hat{s} \leftarrow \mathsf{Recon}(\mathcal{S}, \{\hat{s}_i\}_{i \in \mathcal{S}})$: on input a t-subset $\mathcal{S} \subset [N]$ together with t shares $\{\hat{s}_i\}_{i \in \mathcal{S}}$, outputs a value $\hat{s} \in \hat{S}$.

Definition 4 (Anamorphic Secret Sharing Σ-protocol). *We say that $\mathsf{a}\Sigma$ is an anamorphic secret sharing tuple for Σ if Def. 5, Def. 6, Def. 7, and Def. 8 are satisfied. Similarly, Σ is said to be anamorphic if it admits an anamorphic secret sharing tuple $\mathsf{a}\Sigma$ satisfying that properties.*

As for Σ-protocols, we refer to $\langle \mathsf{aP}(\cdot), \mathsf{V} \rangle(\cdot)$ as the execution of an anamorphic Σ-protocol Σ between a prover aP and a verifier V, where a is computed from P_0 and sent to V, c is sampled at random from C_Σ by V or adversarially chosen if $\mathsf{V} = \mathsf{A}$ and sent to P and finally z is computed by aP using aP_1 and then sent to V. The output of trans will be of the form (a, c, z).

As done in [4] our games will take care of the proper update of the state variables. For this reason also correctness it is more easily formalized via a distinguishing game.

The adversary gets oracle access to O_b for a uniformly sampled bit b. In case $b = 0$, the adversary, querying the oracle with an anamorphic secret $\hat{s}$ and a set of reconstruction indices $\mathcal{S}$, receives back a value which is the result of the share generation and the subsequent reconstruction using the set $\mathcal{S}$ (when $|\mathcal{S}| = t$). If $b = 1$ the oracle simply returns the very same $\hat{s}$ back. In particular, the check of line 10 guarantees our robustness property. That is, in case no share is encoded in a transcript, every party will always realize it.

Definition 5 (Correctness). *$\mathsf{a}\Sigma$ is correct w.r.t a Σ-protocol Σ if for all A, $t, N \in \mathbb{N}, 2 \leq t \leq N$:*

$$\Pr\left[\mathsf{Exp}^{\mathsf{Cor}}_{\mathsf{A},\mathsf{a}\Sigma}(1^\lambda, 1^t, 1^N) = 1\right] \leq \frac{1}{2} + \mathsf{negl}(\lambda)$$

where $\mathsf{Exp}^{\mathsf{Cor}}_{\mathsf{A},\mathsf{a}\Sigma}(1^\lambda, 1^t, 1^N)$ is defined in Fig. 2.

For the following definitions we take inspiration from Sec. 4.1 of [28]. The anamorphic security property requires that the dictator having access to $(x, w, \hat{s})$ but not to the double keys $(\mathsf{dk}, \mathsf{dk}_1, \ldots, \mathsf{dk}_N)$, cannot tell whether it is observing interactions produced by a real prover or by an anamorphic prover. More formally, we introduce the following definition.

$\mathsf{Exp}^{\mathsf{Cor}}_{\mathsf{A},\mathsf{a}\Sigma}(1^\lambda, 1^t, 1^N)$

1 : Sample random $(x, w) \in \mathcal{R}_\mathcal{L}$
2 : $(\mathsf{dk}, \mathsf{dk}_1, \ldots, \mathsf{dk}_N) \leftarrow \mathsf{dKeyGen}(1^\lambda, 1^t, 1^N)$
3 : **for** $i \in [N] : \mathsf{st}_i \leftarrow \epsilon_i$
4 : $b \leftarrow\$ \{0, 1\}$
5 : $b' \leftarrow \mathsf{A}^{\mathsf{O}_b}(1^\lambda, 1^t, 1^N, x, w)$
6 : **if** $b = b'$
7 : **return** 1
8 : **else**
9 : **return** 0

$\mathsf{O}_1(\hat{s}, \mathcal{S})$

1 : **return** $\hat{s}$

$\mathsf{O}_0(\hat{s}, \mathcal{S})$

1 : **if** $\hat{s} \notin \hat{S} \cup \{\perp\} \vee |\mathcal{S}| \neq t \vee \mathcal{S} \not\subset [N]$
2 : **return** $\hat{s}$
3 : $count = 0$
4 : **for** $i \in [N]$:
5 : $\mathsf{tx}_i \leftarrow \mathsf{trans}(\langle \mathsf{aP}(w, \mathsf{dk}, \hat{s}, \mathsf{st}_i), \mathsf{V} \rangle(x))$
6 : $\hat{s}_i \leftarrow \mathsf{ExtShare}(x, \mathsf{tx}_i, \mathsf{dk}_i, \mathsf{st}_i)$
7 : **if** $\perp = \hat{s}_i$
8 : $count++$
9 : $\mathsf{st}_i \leftarrow \mathsf{UpdState}(\mathsf{st}_i)$
10 : **if** $count = N$
11 : **return** $\perp$
12 : **return** $\mathsf{Recon}(\mathcal{S}, \{\hat{s}_i\}_{i \in \mathcal{S}})$

Fig. 2: Correctness Game for Anamorphic Secret Sharing Σ-protocols.

Definition 6 (Anamorphic Security). *$\mathsf{a}\Sigma$ is anamorphic secure w.r.t. a Σ-protocol Σ for a relation $\mathcal{R}_\mathcal{L}$ if, for all PPT A, $t, N \in \mathbb{N}, 2 \leq t \leq N$:*

$$\Pr\left[\mathsf{Exp}^{\mathsf{Ana}}_{\mathsf{A},\Sigma,\mathsf{a}\Sigma}(1^\lambda, 1^t, 1^N) = 1\right] \leq \frac{1}{2} + \mathsf{negl}(\lambda)$$

where $\mathsf{Exp}^{\mathsf{Ana}}_{\mathsf{A},\Sigma,\mathsf{a}\Sigma}(1^\lambda, 1^t, 1^N)$ is defined in Fig. 3.

$\mathsf{Exp}^{\mathsf{Ana}}_{\mathsf{A},\Sigma,\mathsf{a}\Sigma}(1^\lambda, 1^t, 1^N)$

1 : Sample random $(x, w) \in \mathcal{R}_\mathcal{L}$
2 : $(\mathsf{dk}, \mathsf{dk}_1, \ldots, \mathsf{dk}_N) \leftarrow \mathsf{dKeyGen}(1^\lambda, 1^t, 1^N)$
3 : **for** $i \in [N] : \mathsf{st}_i \leftarrow \epsilon_i$
4 : $b \leftarrow\$ \{0, 1\}$
5 : $b' \leftarrow \mathsf{A}^{\mathsf{O}_b(\cdot,\cdot)}(1^\lambda, 1^t, 1^N, x, w)$
6 : **if** $b = b'$
7 : **return** 1
8 : **else**
9 : **return** 0

$\mathsf{O}_0(i, \hat{s})$

1 : **if** $i \notin [N] \vee \hat{s} \notin \hat{S} \cup \{\perp\}$
2 : **return** $\perp$
3 : $\mathsf{tx} \leftarrow \mathsf{trans}(\langle \mathsf{P}(w), \mathsf{A}(1^\lambda, 1^t, 1^N, x, w) \rangle(x))$
4 : **return** tx

$\mathsf{O}_1(i, \hat{s})$

1 : **if** $i \notin [N] \vee \hat{s} \notin \hat{S} \cup \{\perp\}$
2 : **return** $\perp$
3 : $\mathsf{tx} \leftarrow \mathsf{trans}(\langle \mathsf{aP}(w, \mathsf{dk}, \hat{s}, \mathsf{st}_i), \mathsf{A}(1^\lambda, 1^t, 1^N, w) \rangle(x))$
4 : $\mathsf{st}_i \leftarrow \mathsf{UpdState}(\mathsf{st}_i)$
5 : **return** tx

Fig. 3: Anamorphic Security Game for Anamorphic Secret Sharing Σ-protocols.

We model the secrecy notion as a fully adaptive game where the adversary has access to three oracles: a corruption oracle $\mathsf{OCorrupt}(i)$, a share generation oracle $\mathsf{OShare}(\hat{s})$, and a share

$\mathsf{Exp}^{\mathsf{AdSec}}_{\mathsf{A},\mathsf{a}\Sigma}(1^\lambda, 1^t, 1^N)$

1 : Sample random $(x, w) \in \mathcal{R}_\mathcal{L}$
2 : $(\mathsf{dk}, \mathsf{dk}_1, \ldots, \mathsf{dk}_N) \leftarrow \mathsf{dKeyGen}(1^\lambda, 1^t, 1^N)$
3 : Initialize matrix L_{tx}
4 : $\mathcal{C} \leftarrow \emptyset$
5 : **for** $i \in [N] : \mathsf{st}_i \leftarrow \epsilon_i$
6 : $\mathsf{O} \leftarrow \{\mathsf{OCorrupt}, \mathsf{OShare}, \mathsf{OExtShare}\}$
7 : $\hat{s}^0, \hat{s}^1 \leftarrow \mathsf{A}^{\mathsf{O}}(1^\lambda, 1^t, 1^N, x)$
8 : $b \leftarrow\!\!\$\ \{0, 1\}$
9 : $(\mathsf{tx}_1, \ldots, \mathsf{tx}_N) \leftarrow \mathsf{OShare}(\hat{s}^b)$
10 : $i^* \leftarrow \mathsf{L}_{\mathsf{tx}}.\mathsf{length}$
11 : $b' \leftarrow \mathsf{A}^{\mathsf{O}}(\mathsf{tx}_1, \ldots, \mathsf{tx}_N)$
12 : **if** $b' = b \wedge |\mathcal{C}| < t$
13 : **return** 1
14 : **else**
15 : **return** 0

$\mathsf{OShare}(\hat{\mathsf{s}})$

1 : **for** $i \in [N]$:
2 : $\mathsf{tx}_i \leftarrow \mathsf{trans}(\langle \mathsf{aP}(w, \mathsf{dk}, \hat{s}, \mathsf{st}_i), \mathsf{A}(1^\lambda, 1^t, 1^N, w)\rangle(x))$
3 : $\mathsf{st}_i \leftarrow \mathsf{UpdState}(\mathsf{st}_i)$
4 : $\mathsf{L}_{\mathsf{tx}} \leftarrow \mathsf{L}_{\mathsf{tx}}.\mathsf{append}(\mathsf{tx}_1, \ldots, \mathsf{tx}_N)$
5 : **return** $(\mathsf{tx}_1, \ldots, \mathsf{tx}_N)$

$\mathsf{OExtShare}(\hat{s}_i, i, j)$

1 : **if** $\mathsf{L}_{\mathsf{tx}}[i][j] \neq \perp \wedge i \neq i^*$:
2 : **parse** $\mathsf{L}_{\mathsf{tx}}[i][j] = (a, c, z)$
3 : **return** $\mathsf{ExtShare}(x, a, c, z, \mathsf{dk}_j, \mathsf{st})$
4 : **return** $\perp$

$\mathsf{OCorrupt}(i)$

1 : **if** $i \in [N]$:
2 : $\mathcal{C} \leftarrow \mathcal{C} \cup \{i\}$
3 : **return** dk_i
4 : **return** $\perp$

Fig. 4: Adaptive Secrecy Game for Anamorphic Secret Sharing Σ-protocols.

extraction oracle $\mathsf{OExtShare}(\hat{s}, i, j)$. $\mathsf{OCorrupt}$ returns the double key of party i, OShare returns N protocol transcripts resulting from sharing the queried value $\hat{s}$, and $\mathsf{OExtShare}$ returns the extracted share from the j-th transcript of the i-th call to OShare. After a polynomial number of queries to these oracles, A outputs two values and gets N transcripts corresponding to the sharing of a randomly selected one. A wins the game if it can guess the shared valued with probability significantly better than $1/2$, under the condition it did not corrupt more than $t-1$ keys. Notice that before outputting its guess, A is still allowed to perform new sharing queries and to extract the shares for all the secret sharings, except the challenged one.

Definition 7 (Adaptive Secrecy). $\mathsf{a}\Sigma$ *satisfies adaptive secrecy w.r.t. a Σ-protocol Σ for a relation $\mathcal{R}_\mathcal{L}$ if for all PPT* A*, $t, N \in \mathbb{N}, 2 \leq t \leq N$:*

$$\Pr\left[\mathsf{Exp}^{\mathsf{AdSec}}_{\mathsf{A},\mathsf{a}\Sigma}(1^\lambda, 1^t, 1^N) = 1\right] \leq \frac{1}{2} + \mathsf{negl}(\lambda)$$

where $\mathsf{Exp}^{\mathsf{AdSec}}_{\mathsf{A},\mathsf{a}\Sigma}(1^\lambda, 1^t, 1^N)$ *is defined in Fig. 4.*

We define a simulatability property to guarantee that any coalition of less than t receivers learns nothing more about the witness of what it is already leaked by Σ. We do that by requiring the existence of a simulator, which is an ITM interacting the adversary A with inputs the anamorphic secret, the double key of the party and the corresponding states and having oracle access to an ITM running the proving algorithms $\mathsf{P}_0, \mathsf{P}_1$ of Σ with hardcoded statement x and witness w. Finally, the simulator outputs the transcript of its interaction with the adversary. We require that the distribution of the transcripts generated from an execution between an anamorphic prover and the adversary is indistinguishable from the output of the simulator.

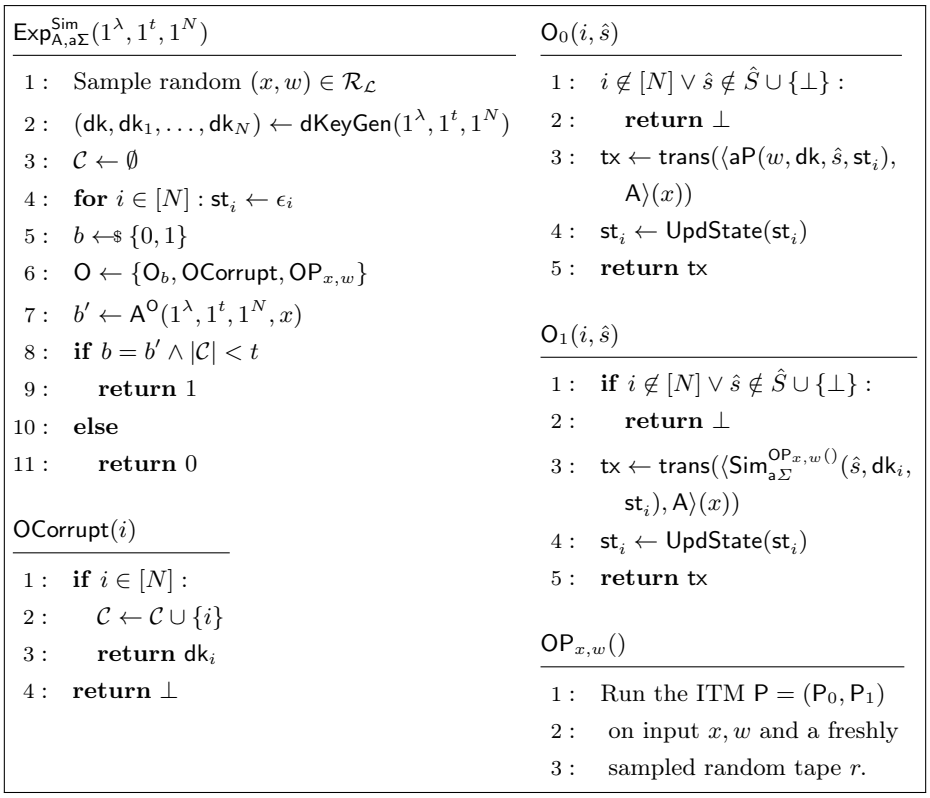

Fig. 5: Simulatability Game for Anamorphic Secret Sharing Σ-protocols.

Definition 8 (Simulatability). $\mathsf{a}\Sigma$ *is simulatable w.r.t* $\Sigma = (\mathsf{P}, \mathsf{V})$ *if there exists a probabilistic interactive oracle machine called simulator* $\mathsf{Sim}^{\mathsf{OP}_{x,w}()}_{\mathsf{a}\Sigma}$ *such that for all PPT* A*, s.t. for all* $t, N \in \mathbb{N}, 2 \leq t \leq N$*:*

$$\Pr\left[\mathsf{Exp}^{\mathsf{Sim}}_{\mathsf{A},\mathsf{a}\Sigma}(1^\lambda, 1^t, 1^N) = 1\right] \leq \frac{1}{2} + \mathsf{negl}(\lambda)$$

where $\mathsf{Exp}^{\mathsf{Sim}}_{\mathsf{A},\mathsf{a}\Sigma}(1^\lambda, 1^t, 1^N)$ *is defined in Fig. 5. Furthermore,* $\mathsf{Sim}_{\mathsf{a}\Sigma}$ *must terminate in an expected polynomial number of steps.*

4.1 Anamorphic Secret Sharing for Pre-Image Σ-Protocols

Let Σ be a pre-image Σ-protocol with associated one-way homomorphism $f : \mathcal{G} \to \mathcal{H}$, and DPRF be a distributed pseudo-random function with key space $\mathcal{K}$, input space $\mathcal{X}$, output and partial output space $\mathcal{G}$, PRF be a pseudo-random function with input space $\mathcal{X}$, key space $\hat{\mathcal{K}}$, and output space $\{0, 1\}$. We construct an anamorphic secret sharing Σ-protocol Σ_{an} as described in Fig. 6. For Σ_{an}, the initial state is $\mathsf{st}_i = (0, i)$.

Theorem 2. *Let* DPRF *be a Distributed PRF with key space* $\mathcal{K}$*, input space* $\mathcal{X}$*, both output and partial output space* $\mathcal{G}$*, enjoying consistency (Def. 2) and adaptive security (Def. 3),* PRF *be a PRF (Def. 17) with output space* $\{0, 1\}$*, and* Σ *be a Pre-Image* Σ*-protocol (Def. 14 and Sec. 3.2) with associated function* $f : \mathcal{G} \to \mathcal{H}$*. Then* Σ_{an} *(Fig. 6) is an Anamorphic Secret Sharing tuple for* Σ *(Def. 4) with anamorphic secret space* $\hat{S} = \mathcal{G}$*.*

Proof. We will prove, in the following lemmas, that Σ_{an} is an anamorphic secret sharing tuple for Σ by proving correctness, adaptive secrecy, and simulatability.

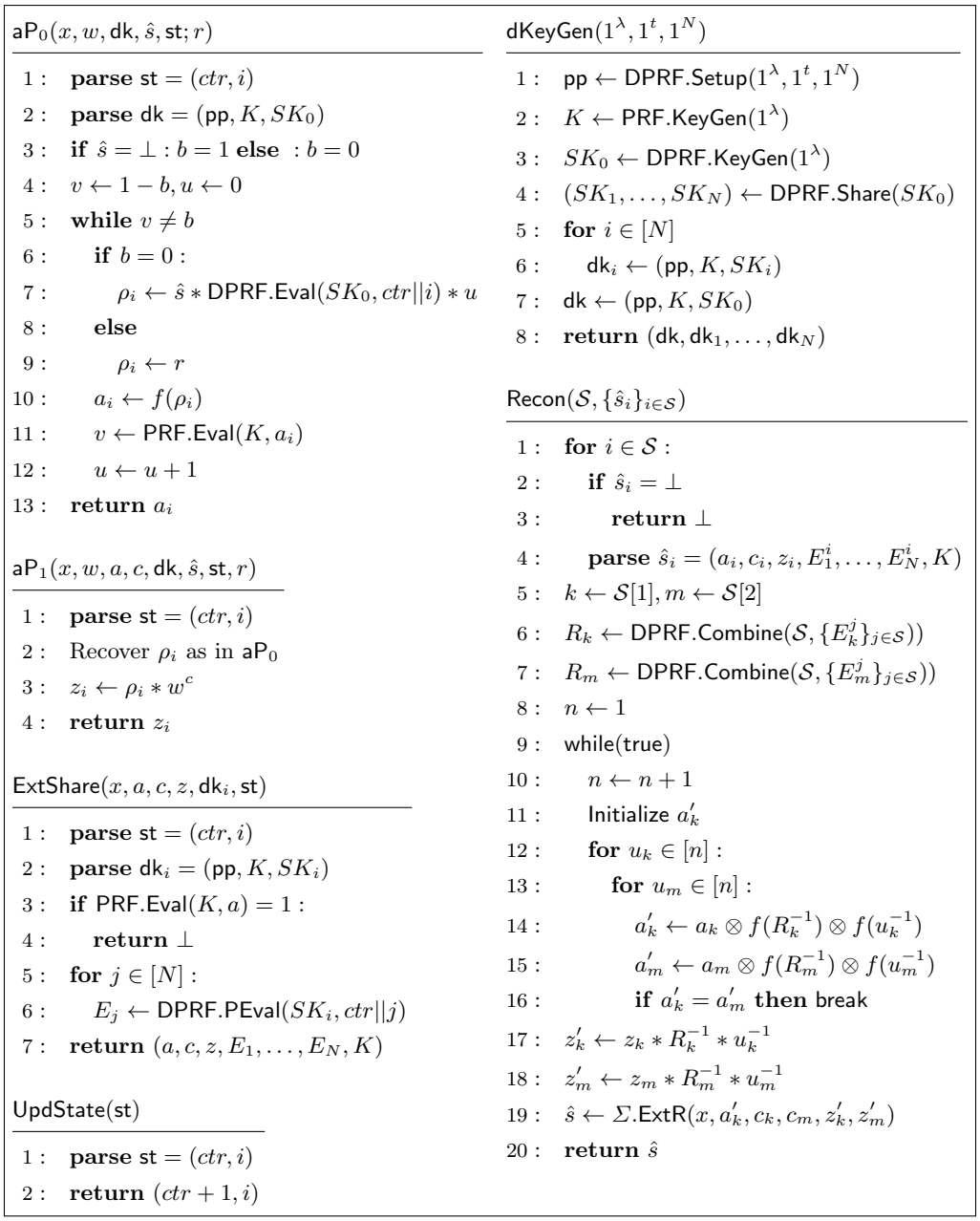

Fig. 6: Anamorphic Secret Sharing tuple Σ_{an} for any Pre-Image Σ-Protocol Σ.

Lemma 1 (Correctness). *If* DPRF *is consistent (Def. 2) and Σ is a pre-image Σ-protocol, then Σ_{an} is correct (Def. 5).*

Proof. First, notice that by consistency of DPRF we have that $\mathsf{Combine}(\mathcal{S}, \{E_i^j\}_{j \in \mathcal{S}}) = \mathsf{DPRF.Eval}(SK_0, \mathsf{st}_i)$ with overwhelming probability. Hence R_i computed using Combine in Recon always matches the value $\mathsf{PRF.Eval}(SK_0, ctr||i)$ inside ρ_i computed by aP$_0$. Also, the special soundness with randomness recoverability of Σ ensures that ExtR will always output the correct secret, since (1) any of the first rounds a_i contained in the t shares received by Recon, when computed correctly by aP$_0$, i.e. $a_i = f(\hat{s} * (R_i = \mathsf{DPRF.Eval}(\mathsf{sk}_0, ctr||i)) * u_i)$ for some state counter ctr and value u such that $\mathsf{PRF.Eval}(K, a_i) = 0$, lead to the same value a when the operation $\otimes f(R_i^{-1}) \otimes f(u_i^{-1})$ is applied to a_i and (2) when the operation $* R_i^{-1} * u_i^{-1}$ is applied to z_i, leading to z'_i, the tran-

script (a, c_i, z'_i) is accepting. This allows the reconstruction of $\hat{s}$ by recovering the randomness (i.e., the input of f) embedded by $a = f(\hat{s} * R_i * u_i * R_i^{-1} * u_i^{-1})$ (that is, the secret $\hat{s}$) through ExtR from two transcripts (a, c_k, z'_k) and (a, c_m, z'_m) for any two indexes k, m included in the reconstruction set $\mathcal{S}$. Notice that the prover aP_0 and the reconstruction algorithm Recon run in expected polynomial time thanks to the pseudorandomness property of PRF. Indeed, the output of $\mathsf{PRF.Eval}(K, a_i)$ will be 0 after 2 expected attempts and hence the correct values u_i for computing a can be calculated by Recon in polynomial time. Finally, notice that if $\hat{s}$ is $\perp$, then the value u is calculated by aP_0 in a way $\mathsf{PRF.Eval}(K, a_i)$ outputs 1. $\mathsf{ExtShare}$, before proceeding with the share extraction, checks if $\mathsf{PRF.Eval}(K, a_i)$ outputs 1, and if that happens, the share $\hat{s}_i$ will be marked as $\perp$. Hence, all the shares computed from $\hat{s} = \perp$ will be marked as $\perp$ and the value *count* in O_0 will match N, leading O_0 to output $\perp$.

Lemma 2 (Adaptive Secrecy). *If* DPRF *is adaptively secure (Def. 3) then* Σ_{an} *satisfies adaptive secrecy (Def. 7).*

Proof. Let us assume that there exists a PPT adversary A breaking the adaptive secrecy property of Σ_{an} with non-negligible advantage, then we construct B breaking adaptive security of DPRF. The proof goes through the following hybrids.

$\mathcal{H}_0$**:** B simulates the adaptive game of Fig. 10 to A, with secret fixed with $\hat{s}_0$.
$\mathcal{H}_1$**:** identical to $\mathcal{H}_0$ except that in $\mathsf{OShare}(\hat{s}^0)$, the prover aP_0, for the index i^* samples a random value instead of computing $\mathsf{DPRF.Eval}(SK_0, ctr||i^*)$.
$\mathcal{H}_2$**:** identical to $\mathcal{H}_1$ to A, except that the secret is fixed with $\hat{s}_1$.
$\mathcal{H}_3$**:** identical to $\mathcal{H}_2$ to A, except that aP_0 computes the first round message a as in the original game, i.e. changes the random value to $\mathsf{DPRF.Eval}(SK_0, ctr||i^*)$.

The proof is concluded by observing that $\mathcal{H}_3$ is identical to the adaptive security game with $b = 1$ and that $\mathcal{H}_0 \approx_c \mathcal{H}_1 \equiv \mathcal{H}_2 \approx_c \mathcal{H}_3$. Let us prove that $\mathcal{H}_0 \approx_c \mathcal{H}_1$ and $\mathcal{H}_2 \approx_c \mathcal{H}_3$.

$\mathcal{H}_0 \approx_c \mathcal{H}_1$**:** B works as follows.
- Upon receiving a $\mathsf{Corrupt}$ query from A for the index i, forward i to the corruption oracle of the adaptive security of the DPRF, receiving back SK_i. Craft a double key dk_i containing SK_i and send dk_i to A.
- Upon receiving a OShare query with input $\hat{s}$, simulate the call to $\mathsf{DPRF.Eval}(SK_0, ctr||i)$ in aP_0 for each tx_i with $i \neq i^*$ by querying $\mathsf{OPEval}(j, ctr||i)$ for all $j \in [N]$ and then use $\mathsf{Combine}$ to obtain $\mathsf{Eval}(SK_0, ctr||i)$. To simulate the first round of tx_{i^*}, query the challenge oracle of the adaptive security of DPRF with $ctr||i^*$, receiving back Y_{i^*}.
- Upon receiving a query to $\mathsf{OExtShare}$ with input $\hat{s}, i, j, \mathsf{st} = (ctr, i)$, retrieve the partial evaluations by querying OPEval with $(j, ctr||k)$ for each $k \in [N]$, receiving back E_k^j. Then, craft the extracted share by including the partial evaluations $E_1^j, \ldots, E_N^j$.
- Finally, output whatever A outputs.

B perfectly simulates $\mathcal{H}_0$ when Y_{i^*} is computed with O_0 and perfectly simulates $\mathcal{H}_1$ when Y_{i^*} is computed randomly by O_1. Thus, assuming there exists a PPT distinguisher D distinguishing between $\mathcal{H}_0$ and $\mathcal{H}_1$ with non-negligible advantage, B can output whatever D outputs and win the DPRF adaptive security game with the same advantage. Indeed, B always behaves as an admissible adversary for $\mathsf{Exp}^{\mathsf{sec}}_{\mathsf{DPRF}}$ since it cannot invoke $\mathsf{Corrupt}$ for more than $t-1$ indexes as required by $\mathsf{Exp}^{\mathsf{sec}}_{\mathsf{DPRF}}$, and cannot ask for any partial evaluations of $(j, ctr||i^*)$ because oracle calls to $\mathsf{OExtShare}$ on i^* are forbidden.
$\mathcal{H}_2 \approx_c \mathcal{H}_3$**:** Works exactly as in $\mathcal{H}_0 \approx_c \mathcal{H}_1$.

Lemma 3 (Anamorphic Security). *If* DPRF *is adaptively secure (Def. 3), then* Σ_{an} *satisfies anamorphic security (Def. 6).*

Proof. We now prove our theorem via a series of indistinguishable hybrids starting from the game with $b = 0$ and ending with the same game with $b = 1$. The proof goes through the following hybrids.

$\mathcal{H}_0$**:** B exactly simulates the anamorphic game of Fig. 3 to A with $b = 0$.
$\mathcal{H}_1$**:** It is identical to $\mathcal{H}_0$ except that P_0 takes a $(x, w, \mathsf{dk}, \hat{s}, \mathsf{st})$ including the state st as done by aP_0, while P_1 takes in input $(x, w, a, c, \mathsf{dk}, \hat{s}, \mathsf{st})$ as for aP_1. Moreover, the prover P parses the input as aP.
$\mathcal{H}_2$**:** It is identical to $\mathcal{H}_1$ except that P_0 initializes b, v, and u as aP_0.
$\mathcal{H}_3$**:** It is identical to $\mathcal{H}_2$ except that if $\hat{s} \neq \perp$, each ρ_i is computed as $\hat{s} * \mathsf{DPRF.Eval}(SK_0, ctr||i) * u$.
$\mathcal{H}_4$**:** It is identical to $\mathcal{H}_3$ except that P_0 computes $v \leftarrow \mathsf{PRF.Eval}(K, a_i)$ and $u \leftarrow u + 1$.
$\mathcal{H}_5$**:** is identical to $\mathcal{H}_4$ except that P_0 executes the while cycle at line 5 of aP_0.

The proof is concluded by observing that $\mathcal{H}_5$ is identical to the anamorphic security game with $b = 1$ and that $\mathcal{H}_0 \equiv \mathcal{H}_1 \equiv \mathcal{H}_2 \approx_c \mathcal{H}_3 \equiv \mathcal{H}_4 \approx_c \mathcal{H}_5$. Let us prove that $\mathcal{H}_2 \approx_c \mathcal{H}_3$ and $\mathcal{H}_4 \approx_c \mathcal{H}_5$.

$\mathcal{H}_2 \approx_c \mathcal{H}_3$**:** notice that a correct execution of the protocol requires that before calling aP_0 the state is updated calling $\mathsf{UpdState}$ with the current state. It will force the increase of ctr causing the generation of different values of a_i after every execution.
Let A be the adversary breaking anamorphic security game with non-negligible advantage, we construct an adversary B that breaks the adaptive security of DPRF with the same advantage. When receiving a challenge oracle query from A with input $(i, \hat{s})$, B queries the challenge oracle of the adaptive security game of the DPRF with $(ctr||i)$, receiving back a value Y_i. Then, B computes $\rho_i = \hat{s} * Y_i * u$ and $a_i = f(\rho_i)$, and sends a_i to A. After receiving c from A, computes $z_i = \rho_i * w^c$ and sends z_i to A. B perfectly simulates the behavior of $\mathcal{H}_2$ when the challenger of adaptive security used O_1 and the behavior of $\mathcal{H}_3$ when the challenger used O_0. Notice that the queries asked by A always induce a B that is an admissible adversary for adaptive security of DPRF. Hence, if there exists a PPT D distinguishing between $\mathcal{H}_2$ and $\mathcal{H}_3$ with non-negligible advantage then B can break adaptive security of DPRF with the same advantage.
$\mathcal{H}_4 \approx_c \mathcal{H}_5$**:** the only difference between the two hybrids is the addition of the while loop. Notice that for simplicity and readability we omitted the following details from the code of aP_0: since the while cycle in line 5 can run indefinitely, aP_0 should take a value $k \in \Omega(\lambda)$ in input that specifies the maximum amount of execution of the loop to do. If a value v that satisfies the condition of the while is reached, aP_0 returns a_i, otherwise aP_0 returns $\perp$. Therefore the two hybrids may be distinguishable since in $\mathcal{H}_4$ aP_0 always returns a_i, while in $\mathcal{H}_5$ it can happen that aP_0 returns $\perp$. Let us assume that in $\mathcal{H}_5$ aP_0 returns $\perp$ with non-negligible probability. Intuitively, if that happens, we can break the real-or-random security of PRF, since this property guarantees that in expected polynomial time the output PRF is b.

Lemma 4 (Simulatability). *If* DPRF *is adaptively secure (Def. 7) and* PRF *satisfies real-or-random security (Def. 17), then* $\mathsf{a}\Sigma$ *is simulatable (Def. 8).*

Proof. We start by defining the interactive Turing machine $\mathsf{Sim}_{\mathsf{a}\Sigma}$ below.

Notice that $\mathsf{Sim}_{\mathsf{a}\Sigma}$ completes within two expected tries. We now prove our theorem via a series of indistinguishable hybrids starting from the simulatability game with $b = 0$ and ending with the same game with $b = 1$.

$\mathcal{H}_0$: It coincides with the simulatability game with $b = 0$

$\mathsf{Sim}^{\mathsf{OP}_{x,w}}_{\mathsf{a}\Sigma}(x, \hat{s}, \mathsf{dk}_i, \mathsf{st}_i)$

1 : $b = 1, \textbf{parse } \mathsf{dk}_i = (\mathsf{p}, K, SK_i)$
2 : **while** $b = 1$:
3 : Call $\mathsf{OP}_{x,w}$ to get first-round message a
4 : $b \leftarrow \mathsf{PRF}(K, a)$
5 : Once received c, send it to $\mathsf{OP}_{x,w}$ to get third-round message z
6 : **return** (a, c, z)

Fig. 7: Simulator for $\mathsf{a}\Sigma$.

$\mathcal{H}_1$: It is identical to $\mathcal{H}_0$ except that aP_0, when $\hat{s} \neq \perp$, uses the random coins r to computes ρ_i instead of using $\mathsf{DPRF.Eval}$. In particular, $\rho_i \leftarrow \hat{s} * r * u$ instead of $\rho_i \leftarrow \hat{s} * \mathsf{DPRF.Eval}(SK_0, ctr||i) * u$.

$\mathcal{H}_2$: It is identical to $\mathcal{H}_1$ except that aP_0, when $\hat{s} \neq \perp$ computes $\rho_i \leftarrow r * u$ ignoring $\hat{s}$.

$\mathcal{H}_3$: It is identical to $\mathcal{H}_1$ except that instead of looping over different values of u, aP_0 is simply re-run with a different randomness r_i, each time setting $\rho_i = r_i$, until $\mathsf{PRF.Eval}(K, a_i) = 0$.

The proof is concluded by observing that $\mathcal{H}_3$ coincided with the simulatability game with $b = 1$ and that $\mathcal{H}_0 \approx_c \mathcal{H}_1 \equiv \mathcal{H}_2 \approx_c \mathcal{H}_3$. $\mathcal{H}_2 \approx_c \mathcal{H}_3$ can be argued as done for Lemma 3. Let us prove that $\mathcal{H}_0 \approx_c \mathcal{H}_1$.

$\mathcal{H}_0 \approx_c \mathcal{H}_1$**:** Let A be an adversary breaking the simulatability game with non-negligible advantage, we can construct an adversary breaking the adaptive security game of DPRF with the same advantage. B forwards any corruption query to the corruption oracle of the adaptive security game of the DPRF. Every time B receives an oracle call to generate a transcript, B computes $\rho_i \leftarrow \hat{s} * Y_i * u$ where Y_i is obtained by querying the evaluation oracle O_b on the current state st_i, then B normally continues its interaction with A. Notice that if $b = 0$, then B perfectly simulates $\mathcal{H}_0$ since Y_i is computed with $\mathsf{DPRF.Eval}(SK_0, \mathsf{st}_i)$, otherwise it perfectly simulates $\mathcal{H}_1$ since Y_i is generated at random. Thus B can output whatever A outputs and win the game with the same advantage. Notice that if A is admissible, B is an admissible adversary for the secrecy game of DPRF since it does not make too many corruption queries or partial evaluation queries.

5 Anamorphic Secret Sharing Signatures

We define anamorphic secret sharing signatures as follows. The definitions for standard anamorphic secret sharing are reported in App. C. An anamorphic secret sharing tuple of poly-time algorithms $\mathsf{ASIG} = (\mathsf{dKeyGen}, \mathsf{aSig}, \mathsf{ExtShare}, \mathsf{Recon})$ w.r.t a signature $\mathsf{SIG} = (\mathsf{KeyGen}, \mathsf{Sign}, \mathsf{Verify})$ is described as follows:

- $(\mathsf{dk}, \mathsf{dk}_1, \ldots, \mathsf{dk}_N) \leftarrow \mathsf{dKeyGen}(1^\lambda, 1^t, 1^N)$: on input the security parameter 1^λ, the threshold 1^t, and the number of parties 1^N, it outputs a master double key dk and individual double key for the receivers $\mathsf{dk}_1, \ldots, \mathsf{dk}_N$.
- $\hat{\sigma} \leftarrow \mathsf{aSig}(\mathsf{sk}, m, \hat{s}, \mathsf{dk}, \mathsf{st})$ takes as input the signing key sk, a *regular* message m, an *anamorphic* message $\hat{s}$, a master double key dk, a state st and outputs an *anamorphic signature* $\hat{\sigma}$.
- $\mathsf{st}' \leftarrow \mathsf{UpdState}(\mathsf{st})$: on input a state st, outputs an updated state st'. The initial state is indicated with the symbol ϵ.
- $\hat{s}_i / \perp \leftarrow \mathsf{ExtShare}(\mathsf{pk}, \hat{\sigma}, \mathsf{dk}_i, \mathsf{st})$: on input a public key pk, a signature $\hat{\sigma}$, a double key of the i-th verifier dk_i, a state st, outputs the i-th share $\hat{s}_i$, or $\perp$ if no share was encoded in the transcript.

– $\hat{s} \leftarrow \mathsf{Recon}(\mathcal{S}, \{\hat{s}_i\}_{i\in\mathcal{S}})$: on input a t-subset $\mathcal{S} \subset [N]$ together with t shares $\{\hat{s}_i\}_{i\in\mathcal{S}}$, outputs a value $\hat{s} \in \hat{S}$.

Definition 9 (Anamorphic Secret Sharing Signatures). *We say that* ASIG *is an anamorphic secret sharing signature w.r.t a signature* SIG *if Def. 10, Def. 11, Def. 12, and Def. 13 are satisfied. Similarly,* SIG *is said to be anamorphic if it admits an anamorphic secret sharing tuple* ASIG *satisfying that properties.*

Definition 10 (Correctness). ASIG *is correct w.r.t to a signature* SIG *if for all* A, $t, N \in \mathbb{N}, 2 \leq t \leq N$*:* $\Pr\left[\mathsf{Exp}^{\mathsf{CorSig}}_{\mathsf{A,SIG,ASIG}}(1^\lambda, 1^t, 1^N) = 1\right] \leq \frac{1}{2} + \mathsf{negl}(\lambda)$ *where* $\mathsf{Exp}^{\mathsf{CorSig}}_{\mathsf{A,SIG,ASIG}}(1^\lambda, 1^t, 1^N)$ *is defined in Fig. 8.*

$\mathsf{Exp}^{\mathsf{CorSig}}_{\mathsf{A,SIG,ASIG}}(1^\lambda, 1^t, 1^N)$

$(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{KeyGen}(1^\lambda)$
$(\mathsf{dk}, \mathsf{dk}_1, \ldots, \mathsf{dk}_N) \leftarrow \mathsf{dKeyGen}(1^\lambda, 1^t, 1^N)$
for $i \in [N]$: $\mathsf{st}_i \leftarrow \epsilon_i$
$b \leftarrow\$ \{0,1\}$
$b' \leftarrow \mathsf{A}^{\mathsf{O}_b}(1^\lambda, 1^t, 1^N, \mathsf{pk}, \mathsf{sk})$
if $b = b'$
  return 1
else
  return 0

$\mathsf{O}_0(\hat{s}, \mathcal{S}, m)$

if $\hat{s} \notin \hat{S} \cup \{\perp\} \vee |\mathcal{S}| \neq t$
  $\vee\, \mathcal{S} \not\subset [N]$:
  return $\hat{s}$
$count = 0$
for $i \in [N]$:
  $\sigma_i \leftarrow \mathsf{aSig}(\mathsf{sk}, m, \hat{s}, \mathsf{dk}, \mathsf{st}_i)$
  $\hat{s}_i = \mathsf{ExtShare}(\mathsf{pk}, \hat{\sigma}_i, \mathsf{dk}_i, \mathsf{st}_i)$
  if $\perp = e_i$
    $count++$
  $\mathsf{st}_i \leftarrow \mathsf{UpdState}(\mathsf{st}_i)$
if $count = N$
  return $\perp$
return $\mathsf{Recon}(\mathcal{S}, \{\hat{s}_i\}_{i\in\mathcal{S}})$

$\mathsf{O}_1(\hat{s}, \mathcal{S}, m)$

return $\hat{s}$

Fig. 8: Correctness Game for Anamorphic Secret Sharing Signatures.

Definition 11 (Anamorphic security). ASIG *satisfies anamorphic security w.r.t to a signature* SIG *if, for all PPT* A, $t, N \in \mathbb{N}, 2 \leq t \leq N$*:*

$$\Pr\left[\mathsf{Exp}^{\mathsf{AnaSig}}_{\mathsf{A,SIG,ASIG}}(1^\lambda, 1^t, 1^N) = 1\right] \leq \frac{1}{2} + \mathsf{negl}(\lambda)$$

where $\mathsf{Exp}^{\mathsf{AnaSig}}_{\mathsf{A,SIG,ASIG}}(1^\lambda, 1^t, 1^N)$ *is defined in Fig. 9.*

Definition 12 (Adaptive Secrecy). ASIG *satisfies adaptive secrecy if for all PPT* A, $t, N \in \mathbb{N}, 2 \leq t \leq N$*:*

$$\Pr\left[\mathsf{Exp}^{\mathsf{AdSecSig}}_{\mathsf{A,SIG,ASIG}}(1^\lambda, 1^t, 1^N) = 1\right] \leq \frac{1}{2} + \mathsf{negl}(\lambda)$$

where $\mathsf{Exp}^{\mathsf{AdSecSig}}_{\mathsf{A,SIG,ASIG}}(1^\lambda, 1^t, 1^N)$ *is defined in Fig. 10.*

$\mathsf{Exp}^{\mathsf{Ana}}_{\mathsf{A},\Sigma,\mathsf{a}\Sigma}(1^\lambda, 1^t, 1^N)$

$(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{KeyGen}(1^\lambda)$
$(\mathsf{dk}, \mathsf{dk}_1, \ldots, \mathsf{dk}_N) \leftarrow \mathsf{dKeyGen}(1^\lambda, 1^t, 1^N)$
for $i \in [N] : \mathsf{st}_i \leftarrow \epsilon_i$
$b \leftarrow\!\$ \{0,1\}$
$b' \leftarrow \mathsf{A}^{\mathsf{O}_b}(1^\lambda, 1^t, 1^N, \mathsf{pk}, \mathsf{sk})$
if $b = b'$
  return 1
else
  return 0

$\mathsf{O}_0(i, \hat{s}, m)$

if $i \notin [N] \vee \hat{s} \notin \hat{S} \cup \{\perp\}$
  return $\perp$
$\sigma \leftarrow \mathsf{Sign}(\mathsf{sk}, m)$
return σ

$\mathsf{O}_1(i, \hat{s}, m)$

if $i \notin [N] \vee \hat{s} \notin \hat{S} \cup \{\perp\}$
  return $\perp$
$\sigma \leftarrow \mathsf{aSig}(\mathsf{sk}, m, \hat{s}, dk, \mathsf{st}_i)$
$\mathsf{st}_i \leftarrow \mathsf{UpdState}(\mathsf{st}_i)$
return σ

Fig. 9: Anamorphic Security Game for Secret sharing Signatures.

$\mathsf{Exp}^{\mathsf{AdSec}}_{\mathsf{A},\mathsf{a}\Sigma}(1^\lambda, 1^t, 1^N)$

$(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{KeyGen}(1^\lambda)$
$(\mathsf{dk}, \mathsf{dk}_1, \ldots, \mathsf{dk}_N) \leftarrow \mathsf{dKeyGen}(1^\lambda, 1^t, 1^N)$
Initialize matrix L_{tx}
$\mathcal{C} \leftarrow \emptyset$
for $i \in [N] : \mathsf{st}_i \leftarrow \epsilon_i$
$\mathsf{O} \leftarrow \{\mathsf{OShare}, \mathsf{OExtShare}, \mathsf{OCorrupt}\}$
$\hat{s}^0, \hat{s}^1 \leftarrow \mathsf{A}^{\mathsf{O}}(1^\lambda, 1^t, 1^N, \mathsf{pk})$
$b \leftarrow\!\$ \{0,1\}$
$(\sigma_1, \ldots, \sigma_N) \leftarrow \mathsf{OShare}(\hat{s}^b)$
$i^* \leftarrow \mathsf{L}_{\mathsf{tx}}.\mathsf{length}$
$b' \leftarrow \mathsf{A}^{\mathsf{O}}(\sigma_1, \ldots, \sigma_N)$
if $b' = b \wedge |\mathcal{C}| < t$
  return 1
else
  return 0

$\mathsf{OShare}(\hat{\mathsf{s}}, \mathsf{m})$

for $i \in [N]$:
  $\sigma_i \leftarrow \mathsf{aSig}(\mathsf{sk}, m, \hat{s}, \mathsf{dk}, \mathsf{st}_i)$
  $\mathsf{st}_i \leftarrow \mathsf{UpdState}(\mathsf{st}_i)$
$\mathsf{L}_{\mathsf{tx}} \leftarrow \mathsf{L}_{\mathsf{tx}}.\mathsf{append}(\sigma_1, \ldots, \sigma_N)$
return $(\sigma_1, \ldots, \sigma_N)$

$\mathsf{OExtShare}(\hat{s}_i, i, j)$

if $\mathsf{L}_{\mathsf{tx}}[i][j] \neq \perp \wedge i \neq i^*$:
  parse $\mathsf{L}_{\mathsf{tx}}[i][j] = \sigma$
  return $\mathsf{ExtShare}(\mathsf{pk}, \sigma, \mathsf{dk}_j, \mathsf{st})$
return $\perp$

$\mathsf{OCorrupt}(i)$

if $i \in [N]$:
  $\mathcal{C} \leftarrow \mathcal{C} \cup \{i\}$
  return dk_i
return $\perp$

Fig. 10: Adaptive Secrecy Game for Anamorphic Secret Sharing Signatures.

Definition 13 (Recipient Unforgeability). ASIG *is recipient unforgeable w.r.t a signature* SIG *if for all* PPT A*,* $t, N \in \mathbb{N}, 2 \leq t \leq N$*:*

$$\Pr\left[\mathsf{Exp}^{\mathsf{ReUnf}}_{\mathsf{A},\mathsf{SIG},\mathsf{ASIG}}(1^\lambda, 1^t, 1^N) = 1\right] \leq \mathsf{negl}(\lambda)$$

where $\mathsf{Exp}^{\mathsf{ReUnf}}_{\mathsf{A},\mathsf{SIG},\mathsf{ASIG}}(1^\lambda, 1^t, 1^N)$ *is defined in Fig. 11.*

5.1 Fiat-Shamir gives Anamorphic Secret Sharing Signatures

The well-known Fiat-Shamir transform compiles any Σ-Protocol $\Sigma = (\mathsf{P}_0, \mathsf{P}_1, \mathsf{V})$ for a relation $\mathcal{R}$ that is secure against passive attacks (Thm. 1) into a signature scheme $\mathsf{SIG} = (\mathsf{KeyGen}, \mathsf{Sign}, \mathsf{Verify})$.

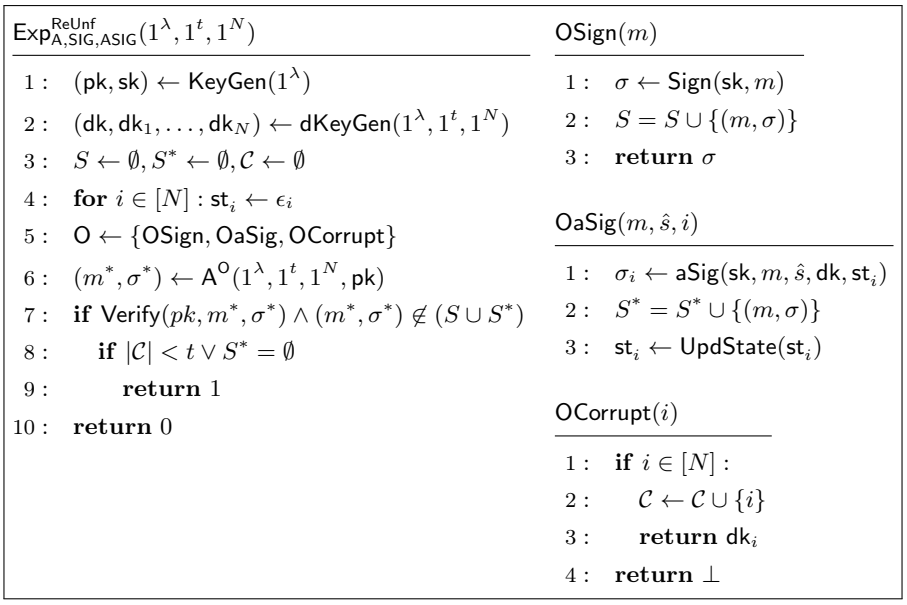

Fig. 11: Recipient Unforgeability Game for Anamorphic Secret Sharing Signatures.

The transform uses an hash function H modeled as a random oracle. Below, with a slight abuse of notation, we identify the relation $\mathcal{R}$ with the sampling algorithm $\mathcal{R}$ that on input 1^λ returns a pair (x, w) with $|x| = \lambda$. In more details, SIG is defined as follows:

- $(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{KeyGen}(1^\lambda)$: $(x, w) \leftarrow \mathcal{R}(1^\lambda)$, return $(\mathsf{pk} = x, \mathsf{sk} = (x, w))$.
- $\sigma \leftarrow \mathsf{Sign}(\mathsf{sk}, m)$: parse $\mathsf{sk} = (x, w)$, $r \leftarrow\$ \mathsf{R}_\Sigma$, $a \leftarrow \Sigma.\mathsf{P}_0(x, w; r)$, $c \leftarrow H(x, a, m)$, $z \leftarrow \Sigma.\mathsf{P}_1(x, w, a, c, r)$ return $\sigma = (a, c, z)$.
- $0/1 \leftarrow \mathsf{Verify}(\mathsf{pk}, m, \sigma)$: parse $\mathsf{pk} = x$ and $\sigma = (a, c, z)$, return 1 iff $c = H(x, a, m)$ and $\mathsf{V}(x, a, c, z) = 1$.

Given an anamorphic tuple $\mathsf{a}\Sigma$ for Σ when can apply the same transformation to $(\mathsf{aP}_0, \mathsf{aP}_1, \mathsf{V})$ to get an anamorphic tuple ASIG for the signature scheme SIG above. In more details, ASIG is defined as follows:

- $(\mathsf{dk}, \mathsf{dk}_1, \ldots, \mathsf{dk}_N) \leftarrow \mathsf{dKeyGen}(1^\lambda, 1^t, 1^N)$: return $\mathsf{a}\Sigma.\mathsf{dKeyGen}(1^\lambda, 1^t, 1^N)$.
- $\sigma \leftarrow \mathsf{aSig}(\mathsf{sk}, m, \hat{s}, \mathsf{dk}, \mathsf{st})$: parse $\mathsf{sk} = (x, w)$,$r \leftarrow\$ \mathsf{R}_\Sigma$, $a \leftarrow \mathsf{aP}_0(x, w, \mathsf{dk}, \hat{s}, \mathsf{st}; r)$, $c \leftarrow H(x, a, m)$, $z \leftarrow \mathsf{aP}_1(x, w, a, c, \mathsf{dk}, \hat{s}, \mathsf{st}, r)$ return $\sigma = (a, c, z)$.
- $\mathsf{st}' \leftarrow \mathsf{UpdState}(\mathsf{st})$: return $\mathsf{a}\Sigma.\mathsf{UpdState}(\mathsf{st})$.
- $\hat{s}_i/\perp \leftarrow \mathsf{ExtShare}(\mathsf{pk}, \sigma, \mathsf{dk}_i, \mathsf{st})$: parse $\mathsf{pk} = x$ and $\sigma = (a, c, z)$ and return $\mathsf{a}\Sigma.\mathsf{ExtShare}(x, a, c, z, \mathsf{dk}_i, \mathsf{st})$.
- $\hat{s} \leftarrow \mathsf{Recon}(\mathcal{S}, \{\hat{s}_i\}_{i \in \mathcal{S}})$: return $\mathsf{a}\Sigma.\mathsf{Recon}(\mathcal{S}, \{\hat{s}_i\}_{i \in \mathcal{S}})$.

Theorem 3. *Let $\mathsf{a}\Sigma$ be an Anamorphic Secret Sharing tuple for Σ (Def. 4). Then, ASIG described above is an Anamorphic Secret Sharing Signature (Def. 9) for SIG in the ROM.*

Proof. We prove the above theorem via different lemmas, one per each required property. We omit correctness as it trivially follows from the correctness of $\mathsf{a}\Sigma$.

Lemma 5 (Anamorphic Security). *If $\mathsf{a}\Sigma$ is anamorphic secure (Def. 6), then ASIG is anamorphic secure (Def. 11).*

Proof. Let us assume for the sake of contradiction that there exists a PPT adversary A breaking anamorphism of ASIG with non-negligible advantage and consider the following PPT adversary B that breaks the anamorphism of $\mathsf{a}\Sigma$. B receives a pair $(x, w) \leftarrow \mathcal{R}(1^\lambda)$ and interacts with an oracle O_b that is either prover P or the anamorphic prover aP. Upon receiving $(1^\lambda, 1^t, 1^N, x, w)$, B sets $(\mathsf{pk}, \mathsf{sk}) = (x, (x, w))$ and runs A on input $(1^\lambda, 1^t, 1^N, \mathsf{pk}, \mathsf{sk})$. Whenever A issues a query, B interacts with O_b to reply to A. In particular, whenever A queries $(i, \hat{s}, m)$, B interacts with its oracle O_b setting as challenge $c = H(x, a, m)$ where a was the first-round message B received from O_b, finally B gets a third-round message z from O_b. B returns the transcript of interaction (a, c, z) with O_b to A. When A stops, B returns the same output returned by A. Since B is perfectly simulating $\mathsf{Exp}^{\mathsf{AnaSig}}_{\mathsf{A},\mathsf{SIG},\mathsf{ASIG}}(1^\lambda, 1^t, 1^N)$ to A, B breaks the anamorphism of $\mathsf{a}\Sigma$ with the same advantage of A.

Lemma 6 (Adaptive Secrecy). *If $\mathsf{a}\Sigma$ satisfies adaptive secrecy (Def. 7), then ASIG satisfies adaptive secrecy (Def. 12).*

Proof. The proof is basically identical to the one above. Assume for the sake of contradiction that there exists a PPT adversary A breaking the adaptive secrecy of ASIG with non-negligible advantage, then we construct PPT B breaking the adaptive secrecy of $\mathsf{a}\Sigma$ with the same advantage. B runs A setting $\mathsf{pk} = x$ and replies to its queries by simply forwarding queries to its own oracles. The only consideration to make is that on receiving an OShare query, B will set $c = H(x, a, m)$ in its interaction with aP. When A stops, B returns the same output returned by A. Since B is perfectly simulating $\mathsf{Exp}^{\mathsf{AdSecSig}}_{\mathsf{A},\mathsf{SIG},\mathsf{ASIG}}(1^\lambda, 1^t, 1^N)$ to A, B breaks the adaptive secrecy of $\mathsf{a}\Sigma$ with the same advantage of A.

Lemma 7 (Recipient Unforgeability). *If SIG satisfies EUF-CMA (Def. 16) and $\mathsf{a}\Sigma$ satisfies simulatability (Def. 8), then ASIG is recipient unforgeable (Def. 13).*

Proof. Let us assume that there exists a PPT adversary A breaking the recipient unforgeability of ASIG with non-negligible probability, then we construct B breaking EUF-CMA of SIG. B runs A simulating the replies to its oracle calls. The proof goes through the following hybrids.

$\mathcal{H}_0$: B exactly simulates the recipient unforgeability game of Fig. 11 to A.

$\mathcal{H}_1$: It is identical to $\mathcal{H}_0$ except B plays in the EUF-CMA game of SIG and sets the public key of the recipient unforgeability game as the one received from the challenger of EUF-CMA. Every time B has to compute $\mathsf{Sign}(\mathsf{sk}, m)$ it queries the signing oracle of EUF-CMA on input m. Furthermore, B replies to the $\mathsf{OaSig}(m, \hat{s}, i)$ queries asked by A using $\mathsf{Sim}_{\mathsf{a}\Sigma}$. In particular, whenever $\mathsf{Sim}_{\mathsf{a}\Sigma}$ makes an oracle call for the first-round message, B just calls the signing oracle of EUF-CMA to produce a signature σ and forwards a taken from σ to $\mathsf{Sim}_{\mathsf{a}\Sigma}$. Whenever $\mathsf{Sim}_{\mathsf{a}\Sigma}$ asks for a challenge c, B sets $c = H(x, a, m)$ and forwards it to $\mathsf{Sim}_{\mathsf{a}\Sigma}$, whenever $\mathsf{Sim}_{\mathsf{a}\Sigma}$ requires the third-round message to its oracle B forwards z. As soon as $\mathsf{Sim}_{\mathsf{a}\Sigma}$ completes an execution with B, B forwards the resulting transcript (a', c', z') to A. Notice that as imposed by the simulatability property, $\mathsf{Sim}_{\mathsf{a}\Sigma}$ returns an (a', c', z') resulting by its interaction with B, i.e., one of the replies of the signing oracle of EUF-CMA game of SIG.

Notice that B in $\mathcal{H}_1$ has no knowledge of sk and thus it is a valid adversary for EUF-CMA of SIG. Assuming A wins recipient unforgeability with non-negligible probability in $\mathcal{H}_1$, then B wins EUF-CMA game of SIG with the same probability. To finish the proof, it suffices to argue that $\mathcal{H}_0 \approx_c \mathcal{H}_1$.

$\mathcal{H}_0 \approx_c \mathcal{H}_1$: Let us assume there exists a PPT distinguisher D that distinguishes between $\mathcal{H}_0$ and $\mathcal{H}_1$ with non-negligible advantage, we can build a PPT adversary C winning the simulatability

game of $\mathsf{a}\Sigma$ with the same advantage. C simulates all the queries asked by A in the following way. Every $\mathsf{OCorrupt}$ query is simply forwarded to the $\mathsf{OCorrupt}$ oracle of the simulatability game. Every OSign query by interacting with $\mathsf{OP}_{x,w}$ setting the challenge c as prescribed by the Fiat-Shamir transform. Finally, it replies to OaSig queries by interacting with O_b setting the challenge c as prescribed by the Fiat-Shamir transform. Notice that C perfectly simulates $\mathcal{H}_0$ is $b = 0$ in the simulatability game and it perfectly simulates $\mathcal{H}_1$ whenever $b = 1$. Notice that all the queries generated by an admissible A generate a set of queries performed by C that are admissible in the simulatability game. In particular, if A did not make more than $t-1$ corruption queries we can rely on simulatability, while if A is admissible but made more than $t-1$ corruption queries it means that it never queried OaSig and we don't have to simulate it at all. C outputs whatever D outputs and wins the simulatability game with the same advantage that D has in distinguishing $\mathcal{H}_0$ from $\mathcal{H}_1$.

6 Conclusions

In this paper, we introduced the notion of Anamorphic Secret Sharing Signatures (ASSS) and showed a construction for signatures based on pre-image Σ-protocols combined with the Fiat-Shamir transform. Our ASSS introduces the possibility of sharing a secret anamorphically and, unlike the Schnorr-based AS of [29], does not reveal the secret key before the anamorphic communication takes place. A future direction involves studying ASSS for post-quantum signatures. A relevant case is CRYSTALS-Dilithium [20, 21] that was recently standardized by NIST [39]. In particular, CRYSTALS-Dilithium is based on Lyubashevky's Σ-protocol [33, 17] combined with the Fiat-Shamir with abort. Lyubashevky's Σ-protocol is a post-quantum Schnorr-Like Σ-protocol with homomorphic properties and thus is a possible candidate for our techniques. However, our current analysis only applies to pre-image Σ-protocols paired with the standard Fiat-Shamir transform.

7 Acknowledgments

This work is supported by the PICOCRYPT project that has received funding from the European Research Council (ERC) under the European Unions Horizon 2020 research and innovation programme (Grant agreement No. 101001283). It was partially supported by projects PRODIGY (TED2021-132464B- I00) and ESPADA (PID2022-142290OB-I00) both funded by MCIN/AEI/10.13039/501100011033/. This work is also part of the grants CEX2024-001471-M funded by MICIU/AEI/10.13039/501100011033, and JDC2023-050791-I funded by the ESF+ and MCIN/AEI/10.13039/501100011033.

This work was also supported by PE11-Made in Italy–Circular and Sustainable (MICS)–European Union Next-Generation-EU (Piano Nazionale di Ripresa e Resilienza–PNRR) and by project SERICS (PE00000014) under the MUR National Recovery and Resilience Plan funded by the European Union—NextGenerationEU.

References

1. Michel Abdalla, Jee Hea An, Mihir Bellare, and Chanathip Namprempre. From identification to signatures via the Fiat-Shamir transform: Minimizing assumptions for security and forward-security. In Lars R. Knudsen, editor, *EUROCRYPT 2002*, volume 2332 of *LNCS*, pages 418–433, Amsterdam, The Netherlands, April 28 – May 2, 2002. Springer Berlin Heidelberg, Germany.
2. Handan Kilinç Alper and Jeffrey Burdges. Two-round trip schnorr multi-signatures via delinearized witnesses. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part I*, volume 12825 of *LNCS*, pages 157–188, Virtual Event, August 16–20, 2021. Springer, Cham, Switzerland.

3. Shalini Banerjee, Tapas Pal, Andy Rupp, and Daniel Slamanig. Simple public key anamorphic encryption and signature using multi-message extensions. Cryptology ePrint Archive, Paper 2025/370, 2025.
4. Fabio Banfi, Konstantin Gegier, Martin Hirt, Ueli Maurer, and Guilherme Rito. Anamorphic encryption, revisited. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part II*, volume 14652 of *LNCS*, pages 3–32, Zurich, Switzerland, May 26–30, 2024. Springer, Cham, Switzerland.
5. Mihir Bellare, Kenneth G. Paterson, and Phillip Rogaway. Security of symmetric encryption against mass surveillance. In Juan A. Garay and Rosario Gennaro, editors, *CRYPTO 2014, Part I*, volume 8616 of *LNCS*, pages 1–19, Santa Barbara, CA, USA, August 17–21, 2014. Springer Berlin Heidelberg, Germany.
6. Mihir Bellare and Phillip Rogaway. The exact security of digital signatures: How to sign with RSA and Rabin. In Ueli M. Maurer, editor, *EUROCRYPT'96*, volume 1070 of *LNCS*, pages 399–416, Saragossa, Spain, May 12–16, 1996. Springer Berlin Heidelberg, Germany.
7. Daniel J. Bernstein, Andreas Hülsing, Stefan Kölbl, Ruben Niederhagen, Joost Rijneveld, and Peter Schwabe. The SPHINCS$^+$ signature framework. In Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, editors, *ACM CCS 2019*, pages 2129–2146, London, UK, November 11–15, 2019. ACM Press.
8. Dan Boneh and Xavier Boyen. Short signatures without random oracles. In Christian Cachin and Jan Camenisch, editors, *EUROCRYPT 2004*, volume 3027 of *LNCS*, pages 56–73, Interlaken, Switzerland, May 2–6, 2004. Springer Berlin Heidelberg, Germany.
9. Dan Boneh, Xavier Boyen, and Hovav Shacham. Short group signatures. In Matthew Franklin, editor, *CRYPTO 2004*, volume 3152 of *LNCS*, pages 41–55, Santa Barbara, CA, USA, August 15–19, 2004. Springer Berlin Heidelberg, Germany.
10. Dan Boneh, Kevin Lewi, Hart William Montgomery, and Ananth Raghunathan. Key homomorphic PRFs and their applications. In Ran Canetti and Juan A. Garay, editors, *CRYPTO 2013, Part I*, volume 8042 of *LNCS*, pages 410–428, Santa Barbara, CA, USA, August 18–22, 2013. Springer Berlin Heidelberg, Germany.
11. Dan Boneh, Ben Lynn, and Hovav Shacham. Short signatures from the Weil pairing. In Colin Boyd, editor, *ASIACRYPT 2001*, volume 2248 of *LNCS*, pages 514–532, Gold Coast, Australia, December 9–13, 2001. Springer Berlin Heidelberg, Germany.
12. Davide Carnemolla, Dario Catalano, Emanuele Giunta, and Francesco Migliaro. Anamorphic resistant encryption: the good, the bad and the ugly. Cryptology ePrint Archive, Paper 2025/233, 2025.
13. Dario Catalano, Emanuele Giunta, and Francesco Migliaro. Anamorphic encryption: New constructions and homomorphic realizations. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part II*, volume 14652 of *LNCS*, pages 33–62, Zurich, Switzerland, May 26–30, 2024. Springer, Cham, Switzerland.
14. Ronald Cramer. *Modular design of secure yet practical cryptographic protocols*. Phd thesis, University of Amsterdam, 1996.
15. Ronald Cramer and Ivan Damgård. Zero-knowledge proofs for finite field arithmetic; or: Can zero-knowledge be for free? In Hugo Krawczyk, editor, *CRYPTO'98*, volume 1462 of *LNCS*, pages 424–441, Santa Barbara, CA, USA, August 23–27, 1998. Springer Berlin Heidelberg, Germany.
16. Amit Deo and Benoît Libert. Anamorphic signatures with dictator and recipient unforgeability for long messages. In Goichiro Hanaoka and Bo-Yin Yang, editors, *Advances in Cryptology - ASIACRYPT 2025 - 31st International Conference on the Theory and Application of Cryptology and Information Security, Melbourne, VIC, Australia, December 8-12, 2025, Proceedings, Part VI*, volume 16250 of *Lecture Notes in Computer Science*, pages 370–401. Springer, 2025.
17. Julien Devevey, Pouria Fallahpour, Alain Passelègue, and Damien Stehlé. A detailed analysis of Fiat-Shamir with aborts. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part V*, volume 14085 of *LNCS*, pages 327–357, Santa Barbara, CA, USA, August 20–24, 2023. Springer, Cham, Switzerland.
18. Xuan Thanh Do, Giuseppe Persiano, Duong Hieu Phan, and Moti Yung. Anamorphism beyond one-to-one messaging: Public-key with anamorphic broadcast mode. In *EUROCRYPT 2025*, 2025.
19. Yevgeniy Dodis and Eli Goldin. Anamorphic-resistant encryption; or why the encryption debate is still alive. Cryptology ePrint Archive, Paper 2025/293, 2025.
20. Léo Ducas, Eike Kiltz, Tancrède Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. Crystals-dilithium: A lattice-based digital signature scheme. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2018(1):238–268, 2018.
21. Léo Ducas, Eike Kiltz, Tancrède Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. Crystals-dilithium. `https://pq-crystals.org/dilithium/`, 2024.
22. Taher ElGamal. A public key cryptosystem and a signature scheme based on discrete logarithms. In G. R. Blakley and David Chaum, editors, *CRYPTO'84*, volume 196 of *LNCS*, pages 10–18, Santa Barbara, CA, USA, August 19–23, 1984. Springer Berlin Heidelberg, Germany.
23. Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In Andrew M. Odlyzko, editor, *CRYPTO'86*, volume 263 of *LNCS*, pages 186–194, Santa Barbara, CA, USA, August 1987. Springer Berlin Heidelberg, Germany.
24. Louis C. Guillou and Jean-Jacques Quisquater. A "paradoxical" indentity-based signature scheme resulting from zero-knowledge. In Shafi Goldwasser, editor, *CRYPTO'88*, volume 403 of *LNCS*, pages 216–231, Santa Barbara, CA, USA, August 21–25, 1990. Springer, New York, USA.

25. Joseph Jaeger and Roy Stracovsky. Dictators? Friends? Forgers. - breaking and fixing unforgeability definitions for anamorphic signature schemes. In Kai-Min Chung and Yu Sasaki, editors, *ASIACRYPT 2024, Part II*, volume 15485 of *LNCS*, pages 105–137, Kolkata, India, December 9–13, 2024. Springer, Singapore, Singapore.
26. Jonathan Katz, Vladimir Kolesnikov, and Xiao Wang. Improved non-interactive zero knowledge with applications to post-quantum signatures. In David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang, editors, *ACM CCS 2018*, pages 525–537, Toronto, ON, Canada, October 15–19, 2018. ACM Press.
27. Chelsea Komlo and Ian Goldberg. FROST: Flexible round-optimized Schnorr threshold signatures. In Orr Dunkelman, Michael J. Jacobson, Jr., and Colin O'Flynn, editors, *SAC 2020*, volume 12804 of *LNCS*, pages 34–65, Halifax, NS, Canada (Virtual Event), October 21-23, 2020. Springer, Cham, Switzerland.
28. Mirek Kutylowski, Giuseppe Persiano, Duong Hieu Phan, Moti Yung, and Marcin Zawada. Anamorphic signatures: Secrecy from a dictator who only permits authentication! *IACR Cryptol. ePrint Arch.*, page 356, 2023.
29. Miroslaw Kutylowski, Giuseppe Persiano, Duong Hieu Phan, Moti Yung, and Marcin Zawada. Anamorphic signatures: Secrecy from a dictator who only permits authentication! In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part II*, volume 14082 of *LNCS*, pages 759–790, Santa Barbara, CA, USA, August 20–24, 2023. Springer, Cham, Switzerland.
30. Miroslaw Kutylowski, Giuseppe Persiano, Duong Hieu Phan, Moti Yung, and Marcin Zawada. The self-anti-censorship nature of encryption: On the prevalence of anamorphic cryptography. *Proc. Priv. Enhancing Technol.*, 2023(4):170–183, 2023.
31. Leslie Lamport. Constructing digital signatures from a one way function. *Technical Report SRI-CSL-98, SRI International Computer Science Laboratory, 1979*, 1979.
32. Benoît Libert, Damien Stehlé, and Radu Titiu. Adaptively secure distributed PRFs from LWE. In Amos Beimel and Stefan Dziembowski, editors, *TCC 2018, Part II*, volume 11240 of *LNCS*, pages 391–421, Panaji, India, November 11–14, 2018. Springer, Cham, Switzerland.
33. Vadim Lyubashevsky. Fiat-Shamir with aborts: Applications to lattice and factoring-based signatures. In Mitsuru Matsui, editor, *ASIACRYPT 2009*, volume 5912 of *LNCS*, pages 598–616, Tokyo, Japan, December 6–10, 2009. Springer Berlin Heidelberg, Germany.
34. Ueli Maurer. Zero-knowledge proofs of knowledge for group homomorphisms. *Des. Codes Cryptogr.*, 77(2-3):663–676, 2015.
35. Silvio Micali and Ray Sidney. A simple method for generating and sharing pseudo-random functions, with applications to clipper-like escrow systems. In Don Coppersmith, editor, *CRYPTO'95*, volume 963 of *LNCS*, pages 185–196, Santa Barbara, CA, USA, August 27–31, 1995. Springer Berlin Heidelberg, Germany.
36. Moni Naor, Benny Pinkas, and Omer Reingold. Distributed pseudo-random functions and KDCs. In Jacques Stern, editor, *EUROCRYPT'99*, volume 1592 of *LNCS*, pages 327–346, Prague, Czech Republic, May 2–6, 1999. Springer Berlin Heidelberg, Germany.
37. Moni Naor and Moti Yung. Universal one-way hash functions and their cryptographic applications. In David S. Johnson, editor, *Proceedings of the 21st Annual ACM Symposium on Theory of Computing, May 14-17, 1989, Seattle, Washington, USA*, pages 33–43. ACM, 1989.
38. Jonas Nick, Tim Ruffing, and Yannick Seurin. MuSig2: Simple two-round Schnorr multi-signatures. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part I*, volume 12825 of *LNCS*, pages 189–221, Virtual Event, August 16–20, 2021. Springer, Cham, Switzerland.
39. NIST. Fips 204 - module-lattice-based digital signature standard. `https://csrc.nist.gov/pubs/fips/204/final`, 2024.
40. Tatsuaki Okamoto. Provably secure and practical identification schemes and corresponding signature schemes. In Ernest F. Brickell, editor, *CRYPTO'92*, volume 740 of *LNCS*, pages 31–53, Santa Barbara, CA, USA, August 16–20, 1993. Springer Berlin Heidelberg, Germany.
41. Giuseppe Persiano, Duong Hieu Phan, and Moti Yung. Anamorphic encryption: Private communication against a dictator. In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part II*, volume 13276 of *LNCS*, pages 34–63, Trondheim, Norway, May 30 – June 3, 2022. Springer, Cham, Switzerland.
42. Giuseppe Persiano, Duong Hieu Phan, and Moti Yung. Public-key anamorphism in (CCA-secure) public-key encryption and beyond. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part II*, volume 14921 of *LNCS*, pages 422–455, Santa Barbara, CA, USA, August 18–22, 2024. Springer, Cham, Switzerland.
43. David Pointcheval and Jacques Stern. Security proofs for signature schemes. In Ueli M. Maurer, editor, *EUROCRYPT'96*, volume 1070 of *LNCS*, pages 387–398, Saragossa, Spain, May 12–16, 1996. Springer Berlin Heidelberg, Germany.
44. Claus-Peter Schnorr. Efficient identification and signatures for smart cards. In Gilles Brassard, editor, *CRYPTO'89*, volume 435 of *LNCS*, pages 239–252, Santa Barbara, CA, USA, August 20–24, 1990. Springer, New York, USA.
45. Yi Wang, Rongmao Chen, Xinyi Huang, and Moti Yung. Sender-anamorphic encryption reformulated: Achieving robust and generic constructions. In Jian Guo and Ron Steinfeld, editors, *ASIACRYPT 2023, Part VI*,

volume 14443 of *LNCS*, pages 135–167, Guangzhou, China, December 4–8, 2023. Springer, Singapore, Singapore.
46. Brent R. Waters. Efficient identity-based encryption without random oracles. In Ronald Cramer, editor, *EUROCRYPT 2005*, volume 3494 of *LNCS*, pages 114–127, Aarhus, Denmark, May 22–26, 2005. Springer Berlin Heidelberg, Germany.

A Deferred Definitions

A.1 Formal Definitions for Σ-Protocols

Definition 14 (Σ-protocol). *A 3-round public-coin protocol $\Sigma = (\mathsf{P}_0, \mathsf{P}_1, \mathsf{V})$, is a Σ-protocol for an NP language $\mathcal{L}$ with a poly-time relation $\mathcal{R}_{\mathcal{L}}$ iff the following properties hold.*

Completeness: *For all x and w such that $(x, w) \in \mathcal{R}_{\mathcal{L}}$ it holds that:*

$$\Pr\left[\mathsf{V}(x, a, c, z) = 1 \;\middle|\; \begin{array}{c} r \leftarrow\!\$\; \mathsf{R}_\Sigma; c \leftarrow\!\$\; \mathsf{C}_\Sigma; \\ a \leftarrow \mathsf{P}_0(x, w; r); \\ z \leftarrow \mathsf{P}_1(x, w, a, c, r) \end{array}\right] = 1.$$

Special Soundness: *There exists a PPT extractor Ext, such that on input x and 2 accepting transcripts $(a, c_0, z_0), (a, c_1, z_1)$ for x, where $c_0 \neq c_1$, it holds that:*

$$\Pr[(x, w) \in \mathcal{R}_{\mathcal{L}} | w \leftarrow \mathsf{Ext}(x, a, c_0, z_0, c_1, z_1)] = 1.$$

Special HVZK (SHVZK): *There exists a PPT algorithm Sim_Σ such that for all $c \in \mathsf{C}_\Sigma$, and all $(x, w) \in \mathcal{R}_{\mathcal{L}}$:*

$$\Big\{(a, c, z) \mid r \leftarrow\!\$\; \mathsf{R}_\Sigma, a \leftarrow \mathsf{P}_0(x, w; r), z \leftarrow \mathsf{P}_1(x, w, a, c, r)\Big\} \approx_c$$
$$\Big\{(a, c, z) \mid (a, z) \leftarrow \mathsf{Sim}_\Sigma(x, c)\Big\}.$$

A.2 Signatures

A signature scheme is a tuple of poly-time algorithms $\mathsf{SIG} = (\mathsf{KeyGen}, \mathsf{Sign}, \mathsf{Verify})$ with message space $\mathcal{M}$, described as follows:

$(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{KeyGen}(1^\lambda)$**:** On input the security parameter, outputs a verification key/signing key pair $(\mathsf{vk}, \mathsf{sk})$.

$\sigma \leftarrow \mathsf{Sign}(\mathsf{sk}, m)$**:** On input a signing key sk and a message $m \in \mathcal{M}$, outputs a signature σ.

$b \leftarrow \mathsf{Verify}(\mathsf{pk}, m, \sigma)$**:** On input a public key pk and a message $m \in \mathcal{M}$, outputs 1 if the signature verifies, and 0 otherwise.

Definition 15 (Correctness). SIG *is correct if, for all $m \in \mathcal{M}$, $\Pr[\mathsf{Verify}(\mathsf{pk}, m, \sigma) = 1]$, where $(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{KeyGen}(1^\lambda)$ and $\sigma \leftarrow \mathsf{Sign}(\mathsf{sk}, m)$.*

Definition 16 (EUF-CMA). SIG *is existentially unforgeable under chosen message attacks if, for all valid PPT adversaries A,*

$$\Pr\left[\mathsf{V}(\mathsf{pk}, m, \sigma) = 1 \;\wedge m \notin \mathcal{Q}_{\mathsf{Sign}}(\mathsf{sk}, \cdot) \;\middle|\; \begin{array}{l} (\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{KeyGen}(1^\lambda); \\ (m, \sigma) \leftarrow \mathsf{A}^{\mathsf{Sign}(\mathsf{sk}, \cdot)}(\mathsf{vk}) \end{array}\right] \leq \mathsf{negl}(\lambda).$$

where $\mathcal{Q}_{\mathsf{Sign}}$ is the set of all the queries made by the adversary to the $\mathsf{Sign}(\mathsf{sk}, \cdot)$ oracle.

A.3 Formal Definition for PRF

Definition 17 (Real-or-random security). *A pseudorandom function $\mathsf{PRF} = (\mathsf{KeyGen}, \mathsf{Eval})$ with keyspace $\mathcal{K}$, input space $\mathcal{X}$ and output space $\mathcal{Y}$. Let Ω be the set of all functions that map inputs in the space $\mathcal{X}$ to outputs in the space $\mathcal{Y}$. The pseudorandom function PRF is called secure if for all PPT adversaries A:*

$$\Big|\Pr[\mathsf{A}^{\mathsf{Eval}(K, \cdot)}(1^\lambda) = 1 \,|\, K \leftarrow\!\$\; \mathsf{KeyGen}(1^\lambda)] - \Pr[\mathsf{A}^{R(\cdot)}(1^\lambda) = 1 \,|\, R \leftarrow\!\$\; \Omega]\Big| \leq \mathsf{negl}(\lambda).$$

B Passive Security of a 3-Round Protocol

Definition 18 (Passive Security). *A 3-round protocol* (P, V) *with challenge space* C *is passively secure for a relation* $\mathcal{R}_{\mathcal{L}}$ *if, for every PPT* A*,*

$$\Pr\left[\mathsf{Exp}^{\mathsf{Pass}}_{\mathsf{A},\mathsf{a}\Sigma}(1^\lambda) = 1\right] \leq \frac{1}{2} + \mathsf{negl}(\lambda)$$

where $\mathsf{Exp}^{\mathsf{Pass}}_{\mathsf{A},\mathsf{a}\Sigma}(1^\lambda)$ *is defined in Fig. 12.*

$\mathsf{Exp}^{\mathsf{Pass}}_{\mathsf{A},\mathsf{a}\Sigma}(1^\lambda)$	$\mathsf{O}_{\mathsf{tx}}()$
1 : Sample random $(x, w) \in \mathcal{R}_{\mathcal{L}}$	1 : $\mathsf{tx} \leftarrow \mathsf{trans}(\langle \mathsf{P}(w), \mathsf{V} \rangle (x))$
2 : $(a, \mathsf{st}) \leftarrow \mathsf{A}^{\mathsf{O}_{\mathsf{tx}}}(x)$	2 : **return** tx
3 : $c \leftarrow \mathsf{C}$	
4 : $z \leftarrow \mathsf{A}(x, a, c, \mathsf{st})$	
5 : **return** $\mathsf{V}(x, a, c, z)$	
6 :	

Fig. 12: Passive Security game for 3-round Protocols (P, V)

C Anamorphic Signatures

We consider the anamorphic signatures definitions in [25].

An anamorphic signature aS is a tuple of poly-time algorithms $(\mathsf{KeyGen}, \mathsf{aSig}, \mathsf{aDec})$ described as follows:

- $(\mathsf{pk}, \mathsf{sk}, \mathsf{dk}) \leftarrow \mathsf{KeyGen}(1^\lambda)$: On input the security parameter, generates a signing key sk, a public key pk, and a double key dk.
- $\hat{\sigma} \leftarrow \mathsf{aSig}(\mathsf{sk}, m, \hat{m}, \mathsf{dk})$: On input a secret key sk, a message m, an anamorphic message $\hat{m}$, and a double key dk, output an anamorphic signature $\hat{\sigma}$.
- $\hat{m} \leftarrow \mathsf{aDec}(m, \hat{\sigma}, \mathsf{dk})$: On input a message m, an anamorphic signature $\hat{\sigma}$, and a double key dk, output the decrypted anamorphic message $\hat{m}$.

Definition 19 (Anamorphic Security). aS *satisfies anamorphic security w.r.t to a signature* SIG *if, for all PPT* A*,*

$$\Pr\left[\mathsf{Exp}^{\mathsf{Ana}}_{\mathsf{A},\mathsf{SIG}}(1^\lambda) = 1\right] \leq \frac{1}{2} + \mathsf{negl}(\lambda)$$

where $\mathsf{Exp}^{\mathsf{Ana}}_{\mathsf{A},\mathsf{SIG}}(1^\lambda)$ *is defined in Fig. 13.*

Definition 20 (Recipient Unforgeability). aS *satisfies recipient unforgeability w.r.t to a signature* SIG *if, for all PPT* A*,:*

$$\Pr\left[\mathsf{Exp}^{\mathsf{ReUnf}}_{\mathsf{A},\mathsf{SIG},\mathsf{aS}}(1^\lambda) = 1\right] \leq \frac{1}{2} + \mathsf{negl}(\lambda)$$

where $\mathsf{Exp}^{\mathsf{ReUnf}}_{\mathsf{A},\mathsf{SIG},\mathsf{aS}}(1^\lambda)$ *is defined in Fig. 14.*

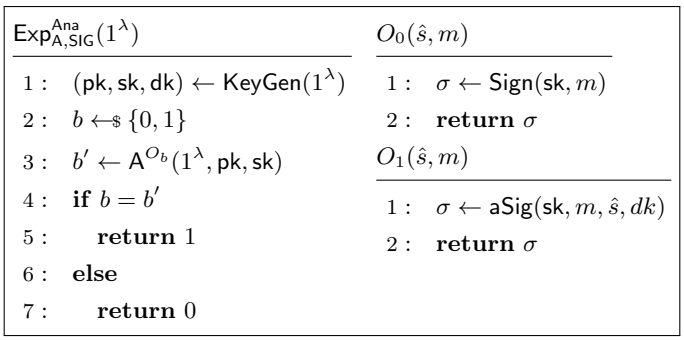

Fig. 13: Anamorphic Security game for Anamorphic Signatures.

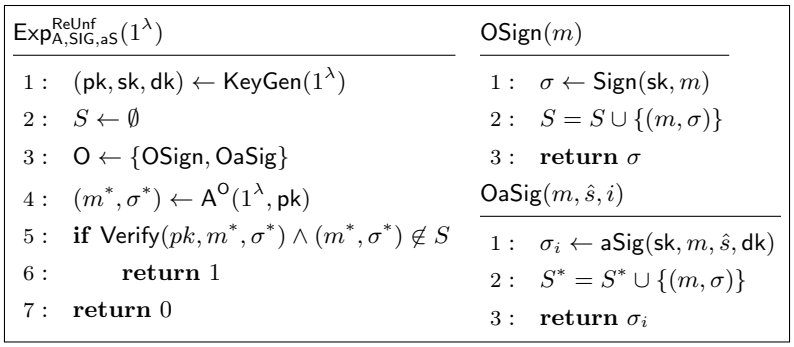

Fig. 14: Recipient Unforgeability for Anamorphic Signatures.